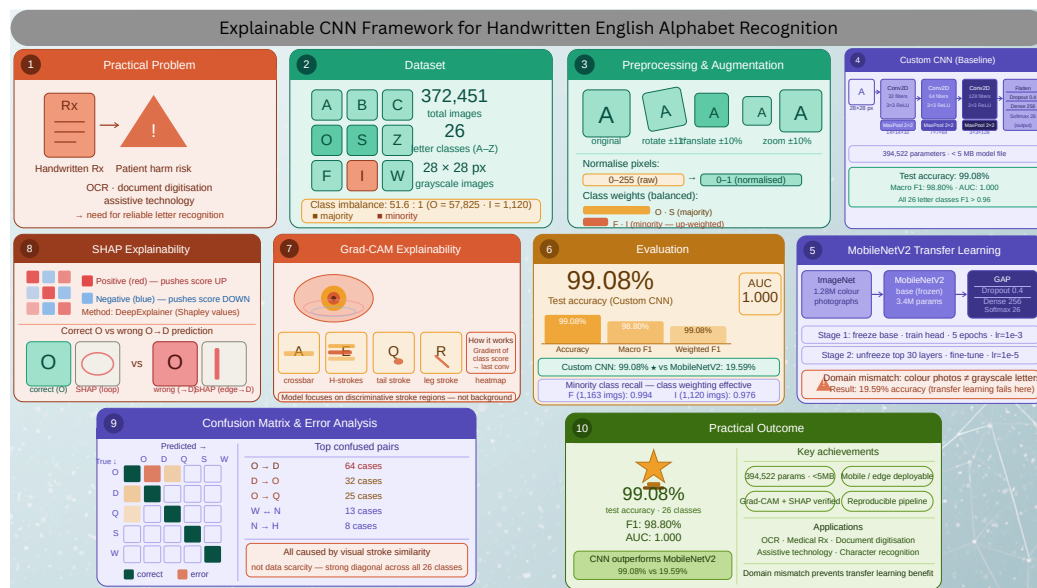


Graphical Abstract

An Explainable CNN-Based Framework for Handwritten English Alphabet Recognition: Comparing Custom CNN and Transfer Learning with Grad-CAM and SHAP

Wai Yan Tun



Highlights

An Explainable CNN-Based Framework for Handwritten English Alphabet Recognition: Comparing Custom CNN and Transfer Learning with Grad-CAM and SHAP

Wai Yan Tun

- A compact custom CNN with 394,522 parameters achieves 99.08% accuracy on 26-class handwritten letter recognition.
- MobileNetV2 transfer learning achieves only 19.59% accuracy due to domain mismatch with grayscale letter images.
- A 51.6:1 class imbalance is handled using stratified splits, augmentation, and balanced class-weighted loss.
- Grad-CAM and SHAP both confirm that the model focuses on meaningful letter stroke regions.
- A fully reproducible and explainable CNN pipeline is provided for the handwriting recognition task.

An Explainable CNN-Based Framework for Handwritten English Alphabet Recognition: Comparing Custom CNN and Transfer Learning with Grad-CAM and SHAP

Wai Yan Tun^a

*^aFaculty of Data Science, University of Europe for Applied Sciences, Think Campus,
Konrad-Zuse-Ring 11, Potsdam, 14469, Brandenburg, Germany*

Abstract

Handwritten character recognition is an important study in computer vision and machine learning. It has many real-world uses, such as reading the medical prescriptions, digitising the documents, and helping people with disabilities who can not write or read properly. Many previous studies used only one model and they did not check whether the model is making correct decisions or not. Most studies also did not even address the differences in the number of images between letter classes, and they did not use two explanation methods together to verify the results too. This study have done everything that the previous studies didn't do by building and comparing two CNN models on a 26-class handwriting dataset and explained the model decisions using both Grad-CAM and SHAP. A custom CNN and a MobileNetV2 transfer learning model are trained on the A-Z Handwritten Alphabets dataset with 372,451 images, a 51.6:1 class imbalance ratio. Class imbalance is handled using three methods; stratified splits, geometric data augmentation, and balanced class-weighted loss. During this study, the custom CNN achieves 99.08% test accuracy, a macro F1-score of 98.80%, and a macro AUC of 0.9999, but MobileNetV2 reaches only 19.59% accuracy. The reason is the domain mismatch between ImageNet photos are colorful images and the images used in the study are grayscale letter images. Both Grad-CAM and SHAP show that the custom CNN model correctly focuses on the ink stroke regions of each letter, and the SHAP comparison between a correct and a wrong prediction reveals why the model confuses O and D. This report give a simple and explainable CNN pipeline that shows transfer

learning does not work everytime. Sometime a lightweight custom models can perform better on small-image handwriting tasks.

Keywords: Convolutional Neural Network, Handwritten Character Recognition, Transfer Learning, Explainable Artificial Intelligence, Gradient-weighted Class Activation Mapping, SHapley Additive exPlanations, Class Imbalance, Image Classification

1. Introduction

1.1. Application Background and Practical Importance

We see handwritten text more often than we think in our daily life, such as medical prescriptions, documents, school exams, postal addresses, and old document records. Even though humans can read them easily, for computers reading them correctly is not that easy. In hospitals and pharmacies, doctors often write prescriptions by hand. And the doctors' handwriting are always hard to read. There is a chance that a nurse will give the wrong medicine or dose to a patient [1]. If this happen, that will be a serious problem for patients in real life. This is one of the many reason how important it is to have a reliable system that can read handwritten letters. A system that can turn this kind of messy human handwriting into clear, readable text would be a lot of help for ordinary people. This is the main goal of Optical Character Recognition (OCR), which converts scanned or photographed text into editable digital text. There are also other organisations that need handwriting recognition, such as postal services that sort letters by address, government offices especially in germany that process in paper forms, and libraries that want to scan and digitalized old books and records. Traditional methods for reading handwritten text used hand-designed features and simple classifiers, but these methods did not work well when writing styles were very different between many people [2]. Some Deep learning methods such as convolutional neural networks (CNN), can learn features directly from pixel data without needing manual design. That is why they are much better at handling different writing styles [3]. Building a reliable system for reading single handwritten letters is the first step for a system that can read full sentences and even the whole book or the whole document.

1.2. Machine Learning and Computer Vision Context

Convolutional neural networks (CNN) are the most popular approach for image classification tasks because as the previous section mention, they learn

30 useful features directly from the pixels of an image [3]. In a CNN model, the
 31 first layers learn simple patterns like edges and curves, and the other layers
 32 combine these into more complex shapes like loops in "O" and crossbars
 33 in "A" that are useful for recognising the letters. This is very useful for
 34 handwriting letter recognition because the shape and direction of strokes
 35 are the most important features for telling letters apart. The pixel-sharing
 36 property of convolution filters makes CNNs efficient for fixed-size images,
 37 and the 28×28 pixel format of the dataset used in this study fits well with
 38 standard CNN designs. A review of deep learning architectures by Alzubaidi
 39 et al. [4] shows that CNNs are the best choice for image tasks because they
 40 remove the need for manual feature engineering and can handle large amounts
 41 of data. One important challenge in character recognition is that the same
 42 letter can look very different because of the person, the writing tool, and
 43 the quality of the scan. Another challenge in this study is that some letters
 44 appear much more often than others in the dataset, which can cause the
 45 model to be biased toward the common letters [5]. But to fix this problem,
 46 this study add a few steps to reduce this bias and makes sure that even the
 47 rarest letters are recognised correctly.

48 *1.3. Transfer Learning for the Selected Problem*

49 Transfer learning is a method where we use a model that was already
 50 trained on a large dataset as the starting point for a new and related task [6].
 51 This means that the model has already learned some useful low-level features
 52 such as edges and textures. These can be reused for the new study without
 53 starting from the beginning. In this study, MobileNetV2 is used as a transfer
 54 learning model. MobileNetV2 is a very popular and efficient architecture
 55 for transfer learning that uses depthwise-separable convolutions to keep the
 56 model small while still achieving a very good accuracy [7]. It was originally
 57 trained on the ImageNet dataset. That dataset contains over one million
 58 colour photographs of everyday objects like cars, animals, and food. One
 59 thing with Transfer learning is, they work better when the source dataset and
 60 the target dataset are similar. If they are very different, the features learned
 61 from the source dataset may not be very useful for the target dataset, and
 62 the model may perform very poorly [8]. In this study, the target dataset is
 63 28×28 grayscale images of handwritten letters on a plain white background,
 64 which is very different from the colour photographs in ImageNet. Testing
 65 whether MobileNetV2 can still perform well with this difference dataset is
 66 one of the main questions of this study.

67 1.4. *Explainable AI and Trustworthiness*

68 A model with a high accuracy score is not always trustworthy, because
69 it might be learning the wrong parts of the image to make its decision. For
70 example, a model might be learning the background colour or image borders
71 instead of learning the actual letter strokes. This would cause problems
72 when the model is used on new images. Explainable AI methods show us
73 to check which parts of an image the model is looking when it makes a
74 prediction [9]. Grad-CAM creates a heatmap that shows which pixels in the
75 last convolutional layer had the most effect when the model is predicting, and
76 then this heatmap is overlaid on the original image so that we can see where
77 the model is focused. SHAP was also used to cross-validate the Grad-CAM
78 results. SHAP assigns a value to each pixel that shows whether that pixel
79 helped or hurt the prediction, with positive values shown in red and negative
80 values shown in blue [10]. Using both methods together is very useful because
81 the way they work are very different. So, if they agree on the same regions,
82 we can be more sure about the model is learning correctly [11]. This study
83 used both Grad-CAM and SHAP not only to check the model is learning
84 correctly but also to understand why the model is making the mistakes.

85 1.5. *Research Gap*

86 Many previous studies that used CNN for handwritten characters recog-
87 nition often have same problems [12]. They tested only one model and do not
88 compare it with a transfer learning model. So, we couldn't know that transfer
89 learning model is better or not. And when studies do the comparison, they
90 rarely report per-class recall, which means we cannot see how well the model
91 performs on rare letter classes. Explainability is also often missing or only
92 includes a few Grad-CAM images for correctly predicted letters. They did
93 not include the misclassified examples or used a second explanation method
94 such as SHAP in this report. Statistical comparison and efficiency analy-
95 sis, such as parameter count and training time, are rarely reported alongside
96 accuracy, making it hard to compare models fairly. The combination of a
97 custom CNN, MobileNetV2 transfer learning, balanced class-weighting, per-
98 class analysis, and dual Grad-CAM and SHAP explainability on a large-scale
99 capital letter dataset has not been reported in the **reviewed literatures**.
100 This study addresses all of these gaps within a single experimental setup.

101 1.6. Problem Statement

102 This study addresses the problem of automatically classifying all 26 hand-
103 written capital English letters from 28×28 grayscale images using a compara-
104 tive and explainable CNN-based framework. The study investigates whether
105 a custom CNN can achieve reliable per-class accuracy on a dataset with
106 large class imbalance, and whether MobileNetV2 transfer learning can im-
107 prove predictive performance over the custom CNN when the source and
108 target domains are very different. The model must identify the correct class
109 label from A to Z from the given 28×28 grayscale image of a handwritten
110 capital letter. There are two key difficulties which is making this project
111 harder than it looks. First, many letters look alike when they are written by
112 hand such as I and L, O and Q, or U and V. So, model can confuse them very
113 easily. Second, the dataset is very imbalanced. Some letters appear ten of
114 thousand of times while others appear only about a thousand times. Because
115 of that model can still get the high overall accuracy while having low accu-
116 racy on the rare letters. Therefore, the project must use both overall and
117 per class metrics to ensure reliable classification. The practical purpose of
118 this system is to make a foundation of the handwritten character recognition
119 applications and use in real world tasks such as reading handwritten medical
120 prescriptions and digitizing paper records.

121 1.7. Aim and Objectives

122 The aim of this study is to develop and evaluate an explainable CNN-
123 based framework for the reliable classification of all 26 handwritten capital
124 English letters from 28×28 grayscale images, with a clear comparison between
125 a custom CNN and a MobileNetV2 transfer learning model using Grad-CAM
126 and SHAP.

127 The objectives are:

- 128 • To analyse and preprocess the A–Z Handwritten Alphabets dataset and
129 handle class imbalance using stratified splits, data augmentation, and
130 class-weighted loss.
- 131 • To build a custom CNN as a baseline classifier for the 26-class task.
- 132 • To implement a MobileNetV2 transfer learning model with two-stage
133 training for comparison.

- 134 • To compare both models using accuracy, macro F1-score, per-class pre-
135 cision and recall, and AUC.
- 136 • To evaluate model efficiency using parameter count and training be-
137 haviour.
- 138 • To explain model predictions using Grad-CAM and SHAP and compare
139 the two methods.
- 140 • To analyse which letter pairs are most often confused and discuss the
141 visual reasons for the errors.

142 1.8. Research Questions

- 143 • **RQ1:** Can a compact custom CNN trained from scratch achieve high
144 overall accuracy on a 26-class handwritten letter recognition task?
- 145 • **RQ2:** Which letter pairs are most often confused, and can visual sim-
146 ilarity at 28×28 pixels explain the errors?
- 147 • **RQ3:** Does the 51.6:1 class imbalance reduce recall on minority letter
148 classes, and do class weighting and augmentation fix this problem?
- 149 • **RQ4:** Does MobileNetV2 transfer learning perform better than the
150 custom CNN on this small grayscale handwriting dataset?
- 151 • **RQ5:** Do Grad-CAM and SHAP confirm that the model focuses on
152 letter stroke regions rather than background pixels?

153 1.9. Contributions and Novelty

154 This study is not just using a standard CNN to a handwriting dataset.
155 It compares custom CNN and MobileNetV2, handle the class imbalance,
156 class-wise reliability analysis, and uses Grad-CAM and SHAP methods, all
157 within a single reproducible pipeline. Each contribution below addresses the
158 limitations found in the literature review and is supported by experimental
159 results.

- 160 • **Systematic comparison of a custom CNN and transfer learn-**
161 **ing.** A custom CNN with 394,522 parameters and a MobileNetV2
162 transfer learning model are trained and tested on the same data split

under the same conditions. The results show that ImageNet pre-training is not that helpful when the input images are in grayscale. This gives useful information to researchers who need to decide between a custom model and a pre-trained model.

- **Class-wise reliability analysis under large class imbalance.** The 51.6:1 imbalance between the most and least common letters is handled using 3 methods; stratified splitting, data augmentation, and balanced class-weighted loss. After the model training, per-class precision, recall, and F1-score are reported for all 26 letters. They show that even the minority class recall stays above 0.976.
- **Dual explainability using Grad-CAM and SHAP.** Both Grad-CAM and SHAP are used to the custom CNN and their results are compared. When the both methods show to the same stroke regions, the evidence that the model is reasoning correctly is much stronger than if only one method was used. The SHAP comparison between a correct "O" prediction and a wrong "O" prediction also explains exactly why the model confuses "O" with "D".
- **Fully reproducible pipeline.** The complete pipeline from data loading to Grad-CAM and SHAP is written in a single public Kaggle notebook (<https://www.kaggle.com/code/waiyantun17/handwritten-english-alphabet-recognition-using-cnn>). All figures and result tables are saved as PDF and CSV files so every number in this report can be checked independently.

1.10. Report Organization

The rest of this report is structured as follows. Section 2 reviews related literature on CNN-based character recognition, transfer learning, data augmentation, evaluation methods, and explainable AI. Section 3 describes the methodology including the research design, the dataset, preprocessing, model design, training, evaluation, and explainability methods. Section 4 shows the experimental results organised by research question. Section 5 discusses the findings and compares them to the literature. Section 6 concludes with a summary of the contributions, limitations, and ideas for future work.

195 2. Literature Review

196 2.1. Image-Based Machine Learning in the Application Domain

197 Handwritten character recognition has been studied for many years. it
198 started with simple template matching and then rule-based methods, and
199 now they are moving to statistical classifiers and deep learning [3]. Earlier
200 methods used hand-crafted features such as pixel pattern, stroke directions,
201 and loop counts to describe each character. These methods needed a lot of
202 manual work to design the features, and they did not perform well to new
203 writers or difference writing styles [2]. The development of large labelled
204 datasets such as MNIST for digits and EMNIST for letters gave researchers
205 a standard way to evaluate and compare different methods [13]. Unlike earlier
206 methods, Deep learning models can learn important features automatically
207 from the raw pixel data, without manual feature engineering. A comprehensive
208 review by Alzubaidi et al. [4] shows that deep learning, and CNNs, have
209 become the leading approach for image recognition because of their ability to
210 learn hierarchical representations from data. The main practical challenge
211 in handwriting recognition is that the same letter can look very different
212 between writers. And even the same writer may have written the slightly
213 different letter every time. Therefore, having a large and diverse training
214 dataset is important, and techniques such as data augmentation should be
215 used to create more varied examples during training.

216 2.2. CNN-Based Methods for the Selected Task

217 LeCun et al. [3] showed that CNNs could achieve high accuracy on digit
218 recognition tasks by learning features directly from pixel data, which was an
219 important step that led to the wide use of CNNs in character recognition.
220 Cohen et al. [13] extended the MNIST dataset to include letters and created
221 EMNIST, which is a standard benchmark for handwritten letter recognition
222 with 62 classes covering digits, uppercase, and lowercase letters. Baldominos
223 et al. [12] carried out a systematic comparison of different CNN designs on
224 MNIST and EMNIST, and found that networks with batch normalisation
225 and dropout performed better than simpler designs, but very deep networks
226 did not give much extra improvement on 28×28 images. Deng et al. [14]
227 studied deep learning approaches for handwritten Chinese character classification
228 and found that compact CNN designs with careful preprocessing can
229 achieve strong recognition accuracy even on low-resolution character images.
230 Ahlawat and Choudhary [15] tested a hybrid CNN and SVM approach for

handwritten digit recognition and found that moderate-depth CNNs with dropout gave a good balance between accuracy and model size. A common weakness or limitation in many of these studies is that they report only overall accuracy and did not look at per-class performance, which makes it hard to see how the model handles rare classes or visually similar letter pairs. This study fills this gap by reporting full per-class metrics and by looking at how the imbalance between classes affects the recall of less frequent letters.

2.3. *Transfer Learning Architectures*

Transfer learning has become widely used in image classification because it allows a model that was already trained on a large dataset to be reused for a new, smaller task without starting from the beginning [6]. MobileNetV2, introduced by Sandler et al. in 2018 [7], is the transfer learning model used in this study. It uses inverted residual blocks with linear bottlenecks and depthwise separable convolutions, which make the model small and fast while keeping a good accuracy even compare to much larger models. MobileNetV2 has around 3.4 million parameters in its base and was designed to use on mobile and embedded devices with limited memory and processing power. It was pre-trained on ImageNet, which contains 1.28 million colour photographs across 1,000 classes, making it a strong feature extractor for natural image recognition. However, MobileNetV2 expects colour input images of at least 32×32 pixels, which means the 28×28 grayscale images in this study must be resized and converted to pseudo-RGB before they can be used for the model. This difference between the original ImageNet data and the dataset in this study may reduce the accuracy of the pre-trained features. Other architectures such as ResNet50 [16], DenseNet [17], and EfficientNet [18] are not used in this study but are noted as alternatives that have been successfully applied to image classification tasks in related work. A study by Abou Baker et al. [8] evaluated eleven pre-trained models on five different target datasets and found that the best model depends strongly on how similar the target domain is to the source domain, which is a key concern for this study. Mahbod et al. [19] also highlighted that domain mismatch is a major reason why transfer learning sometimes fails to help. These findings motivates the comparison between MobileNetV2 and the custom CNN in this study.

265 2.4. Data Preprocessing and Augmentation

266 Data preprocessing and augmentation are very important steps in train-
267 ing a CNN, especially when the training data is not perfectly balanced [20].
268 Pixel normalisation scales pixel values from the range $[0, 255]$ to $[0, 1]$. This
269 helps to keep the stable size gradients during training and is a standard step
270 in all CNN training pipelines. Geometric augmentation operations such as
271 rotation, translation, and zoom in or out create new versions of each image
272 that look slightly different from the origin. This helps the model to recognise
273 the letters that are tilted, shifted, or slightly bigger or smaller. Shorten and
274 Khoshgoftaar [20] reviewed many image classification studies and found that
275 geometric augmentation can improved the model generalization, with rota-
276 tion and translation being the most effective. For handwriting recognition,
277 rotation should be kept to small angles such as ± 10 to 15 degrees, because
278 large rotations can make one letter look like a different letter. Horizontal
279 flipping is not suitable for letter recognition because flipping a letter creates
280 a mirror image that does not represent to the same letter. Class imbalance
281 is a separate problem that augmentation alone cannot fully solve, especially
282 when the imbalance ratio is as large as 51.6:1 as in this dataset. Class-
283 weighted loss is one of the best way to address this by giving more weight to
284 errors on rare classes during training. Johnson and Khoshgoftaar [5] showed
285 that combining augmentation with cost-sensitive learning gives better results
286 than using only one method.

287 2.5. Model Evaluation and Reliability

288 Evaluating an image classification model requires more than just report-
289 ing overall accuracy, especially when the classes are not equally balanced [21].
290 If the dataset has many more images of some classes than others, a model
291 might always predict the most common class with high accuracy, and fail
292 completely on the rare classes. Grandini et al. [21] studied different evalua-
293 tion metrics for multi-class classification and recommended reporting macro-
294 average F1-score alongside weighted-average F1-score and per-class results
295 to give a complete evaluation of the model performance. Macro-average F1-
296 score treats all classes equally no matter how many images they have, which
297 makes it a better metric for imbalanced datasets. The area under the re-
298 ceiver operating characteristic curve (AUC) is another useful metric because
299 it measures how well the model separates one class from all others, with-
300 out needing to choose a decision threshold. Per-class precision and recall
301 are also important because they show which specific letters are hardest to

302 recognise and why, which is useful information for improving the model. A
303 common problem in the literature is that many studies only report accuracy
304 and do not provide per-class metrics or confusion matrices, which makes it
305 impossible to understand where the model fails [12]. This study reports ac-
306 curacy, macro F1, weighted F1, per-class precision, recall and F1, AUC for
307 all 26 classes, and a normalised confusion matrix to provide a full and fair
308 evaluation.

309 2.6. Explainable AI in Image Analysis

310 2.6.1. Grad-CAM

311 Grad-CAM was introduced by Selvaraju et al. in 2017
312 citeSelvaraju2017. It is a visual explanation method that shows which parts
313 of an image are most important for a model’s prediction. It calculates gra-
314 dients for each feature channel, averages them to get a weight, and then
315 combines the weighted feature maps to produce a heatmap. This heatmap
316 is resized to the same size as the input image and overlap on it, to see which
317 part of the image is used to make the prediction. Grad-CAM has been used
318 in many medical image studies to check whether a model is looking at the
319 right part of an image, such as a tumour or a lesion [22]. In handwriting
320 recognition, it can show whether the model is focusing on the ink strokes of
321 the letter or on the background.

322 2.6.2. SHAP

323 SHapley Additive exPlanations also known as SHAP was introduced by
324 Lundberg and Lee in 2017[10]. It’s based on Shapley values from game the-
325 ory. For image classification, each pixel is treated as one feature, and its
326 SHAP value shows how much that pixel helped or hurt the predicted class
327 score. A positive SHAP value means the pixel pushed the predicted proba-
328 bility higher, and a negative value means it pushed it lower. DeepExplainer
329 is the deep learning version of SHAP, and it works by sending the Shapley
330 values backward through the network layers using a modified version of back-
331 propagation. Zahedipour et al. [23] used SHAP on a CNN for Alzheimer’s
332 disease classification and showed that it can clearly identify which parts of a
333 brain scan are most important for the prediction. Akter et al. [24] combined
334 SHAP with Grad-CAM in a COVID-19 diagnosis study and found that using
335 both methods together gave a better understanding of model behaviour than
336 using only one.

2.6.3. Explainability Limitations

Even though Grad-CAM and SHAP create the useful visual maps, that doesn't mean these the model is making decision with correct reasons [11]. Adebayo et al. [11] showed that some explanation methods produce similar-looking maps even when the model is not properly trained. This means that the maps can sometimes look correct but may not represent the model's decision process. Grad-CAM also has some limitations. It uses feature maps from the last convolutional layer, which have lower resolution than the original image. When these maps are enlarged to match the input image size, some small details may be lost. Chattopadhyay et al. [25] introduced Grad-CAM++ to improve the quality of the explanations when there are multiple examples of the same object in one image. SHAP also has limitations because it depends on a background dataset for comparison. If the selected background images are not similar to the training data, the explanation results may not correctly show the importance of each pixel. The explanation results can be changed depend on the model architecture, the training seed, and the choice of background images. This study reduces this risk by applying both Grad-CAM and SHAP and checking whether they show to the same regions or not. The explanations are also compared between correctly and incorrectly classified images to understand why the model makes certain predictions.

2.7. Robustness, Generalization, and Deployment

A model that works well on a clean test dataset may not work as well as in real-world data where images can be blurry, noisy, or taken under poor lighting [26]. Low-resolution images like 28×28 pixels are especially sensitive to these small changes, because even a small change in a few pixels can change the appearance of a character. Nisar et al. [26] found that models need to be adjected when the conditions between training and real-world data are different, and this is especially important for handwriting recognition where scanning conditions can be different. Mukhlif et al. [27] suggested a dual transfer learning method for medical images to reduce the problem of domain mismatch, which is the same problem that MobileNetV2 faces in this study. The custom CNN in this study has fewer than 400,000 parameters, which is small enough to run on a mobile phone or embedded device where memory and processing power are limited. Cheng et al. [28] showed that transfer learning across different domains can work only when the two domains look similar enough, which is also not true for this study. Testing how well the

374 model handles noisy or blurry images is not done in this study, but it is
375 an important step that should be done before the model is used in a real
376 application.

377 *2.8. Comparative Literature Review Matrix*

378 Table 1 shows a comparison of 15 closely related peer-reviewed stud-
379 ies alongside the proposed work in this study. Looking at the table, it is
380 clear that most studies use only one model and do not compare it with a
381 transfer learning baseline. SHAP is rarely used in handwriting recognition
382 studies, and no reviewed study applies both Grad-CAM and SHAP together
383 on this type of dataset. Class imbalance treatment is often missing or only
384 partially applied, and external validation on a separate dataset is almost
385 never included. The proposed study is the only one in the table that com-
386 bines multiple-model comparison, rigorous imbalance treatment, per-class
387 reliability analysis, both Grad-CAM and SHAP explanations, and efficiency
388 reporting within a single reproducible pipeline.

389 *2.9. Synthesis of Literature and Identified Gap*

390 The reviewed literature shows that CNNs are widely used for handwritten
391 character recognition, and even a small CNN model can preform well on small
392 fixed-size images [12, 13]. Data augmentation is the most commonly used tool
393 for handling various writing styles, and Grad-CAM is the most frequently
394 applied explainability method among all the reviewed studies. However,
395 findings are different about transfer learning helps for small-image tasks, with
396 some studies reporting clear gains [8] and others found only little benefit when
397 the source and target domains are different [19]. What is missing is a study
398 that combines model comparison, great class imbalance handling, detailed
399 per-class evaluation analysis, and more than one explainability method in
400 one reproducible pipeline on a large-scale capital letter dataset. There is
401 almost no reviewed study compares Grad-CAM and SHAP side by side on
402 this type of data. This study is designed to fill all these gaps, and the
403 methodology described in the next section explains how each component of
404 the pipeline was built and evaluated.

405 **3. Methodology**

406 *3.1. Research Design*

407 This study follows a quantitative experimental design in which two CNN
408 architectures are trained and evaluated under the same conditions on the

Table 1: Comparative literature review matrix summarising datasets, models, evaluation practices, explainability methods, and limitations across 15 closely related image classification studies. The final row shows the proposed study. ✓ = feature included; ✗ = not included; ~ = partially included. TL = Transfer Learning, CB = Class Balancing, MC = Multiple-Model Comparison, GC = Grad-CAM, SH = SHAP, RT = Robustness Testing, EV = External Validation, ST = Statistical Testing, EA = Efficiency Analysis.

Study	Problem	Dataset	Classes	Size	Models	TL	CB	MC	GC	SH	RT	EV	ST	EA	Metrics	Main Contribution	Principal Limitation
LeCun et al. [8], 1998	Digit recognition	MNIST	10	70,000	LeNet-5	✗	✗	✗	✗	✗	✗	✗	✗	✗	Accuracy	First CNN for document recognition	No letters; no explainability
Cohen et al. [13], 2017	Alphanumeric recognition	EMNIST	62	814,255	CNN variants	✗	~	✓	✗	✗	✗	✗	✗	✗	Accuracy, F1	Extended MNIST to letters	No explainability; partial imbalance
Baldominos et al. [12], 2019	Letter recognition	EMNIST	26	145,600	Multiple CNNs	✗	✗	✓	✗	✗	✗	✗	✗	~	Accuracy	Evolutionary gradient CNN	No explainability; no class-level analysis
Deng et al. [14], 2024	Handwritten Chinese character recognition	Custom dataset	3755	300,000+	Deep CNN	✗	~	~	✗	✗	✗	✗	✗	~	Accuracy, F1	Deep CNN for low-res character classification	No explainability; Chinese script only
Ahluwat & Chhabra [15], 2020	Digit recognition	MNIST	10	70,000	CNN-SVM	✗	~	~	✗	✗	✗	✗	✗	✗	Accuracy	Hybrid CNN-SVM classifier	No explainability; digits only
Chauhan et al. [29], 2024	Multi-script character classification	Multiple	Various	Various	HCR-Net	✗	~	~	✗	✗	✗	✗	✗	✓	Accuracy, F1	Script-independent CNN	No explainability; no SHAP
Saxena et al. [7], 2018	Image classification	ImageNet	1000	1.28M	MobileNetV2	✓	✗	✓	✗	✗	✗	✗	✗	✓	Top-1, Top-5	Inverted residuals for efficient TL	Not applied to handwriting
Ho et al. [16], 2020	Image classification	ImageNet	1000	1.28M	ResNet50	✓	✗	✓	✗	✗	✗	✗	✗	✓	Top-1, Top-5	Residual connections	Not applied to handwriting
Selvaraju et al. [9], 2017	Visual explanation	Multiple	Various	Various	Grad-CAM	✓	✗	✓	✓	✗	✗	✗	✗	✗	Localisation	Gradient-based visual explanation	Coarse localisation; no SHAP
Lundberg & Lee [10], 2017	Model explanation	Multiple	Various	Various	SHAP	✗	✗	✓	✗	✓	✗	✗	✗	✗	Attribution	Shapley-based feature attribution	Computationally expensive
Abou Baker et al. [8], 2022	Transfer learning eval	Multiple	Various	Various	11 CNN models	✓	✗	✓	✗	✗	✗	✗	✗	✓	Accuracy	Evaluation of 11 pre-trained models	No explainability; no handwriting
Cheng et al. [28], 2023	Remote sensing	RS dataset	Various	Various	ResNet, VGG	✓	✗	✓	✗	✗	✗	✗	✗	~	Accuracy	Cross-domain transfer learning	No explainability; different domain
Sherpa & Pant [22], 2025	Blood image classification	BloodMNIST	8	17,092	Multiple CNNs	✓	~	✓	✓	✓	✗	✗	✗	✓	Accuracy, AUC	Explainable AI with dynamic model selection	Not applied to handwriting
Akter et al. [24], 2024	COVID-19 diagnosis	COVIDx	2-3	29,986	Ensemble CNNs	✓	~	✓	✓	✓	✓	✓	✗	~	Accuracy, AUC	Ensemble XAI for COVID-19	Binary task; not handwriting
Narayan et al. [26], 2023	Domain adaptation	Office-Home	Various	Various	UDA-CNN	✓	✗	✓	✗	✗	✓	✓	✗	~	Accuracy	Domain adaptation with JS divergence	No explainability; not handwriting
Proposed study	26-class letter recognition	A-Z Kagle	26	372,451	Custom CNN, MobileNetV2	✓	✓	✓	✓	✓	✓	✗	✗	✓	Acc, AUC, per-class	Dual XAI, imbalance treatment, custom CNN vs TL	Single dataset; no external validation

409 same dataset split. The task is 26-class image classification, where each
 410 input image must be assigned to one of 26 capital letter categories. The
 411 study is comparative because a custom CNN is directly compared with a
 412 MobileNetV2 transfer learning model using the same training data, test set,
 413 and evaluation metrics. Explainability is treated as an equal part of the
 414 experimental design, not as an extra step added at the end. Both Grad-
 415 CAM and SHAP are applied to the custom CNN after training to check
 416 whether the model is using the correct image regions and to understand why
 417 it makes mistakes. All experiments are run in a Kaggle Notebook with GPU
 418 acceleration, and the notebook is made public so all results can be reproduced
 419 by others. The full experimental workflow is shown in Fig. 1.

420 *3.2. Dataset Description*

421 *3.2.1. Dataset Source and Accessibility*

422 The A–Z Handwritten Alphabets in CSV Format dataset was published
 423 on Kaggle by Sachin Patel and is freely available at [https://www.kaggle](https://www.kaggle.com/datasets/sachinpatel21/az-handwritten-alphabets-in-csv-format)
 424 [.com/datasets/sachinpatel21/az-handwritten-alphabets-in-csv-f](https://www.kaggle.com/datasets/sachinpatel21/az-handwritten-alphabets-in-csv-format)
 425 [ormat](https://www.kaggle.com/datasets/sachinpatel21/az-handwritten-alphabets-in-csv-format). The dataset has a public domain licence and does not contain any
 426 personal information about the people who wrote the letters. It was chosen
 427 for this study because it is large, well-organised, and in a format that can be
 428 read directly into a CNN training pipeline.

429 *3.2.2. Dataset Composition*

430 The dataset is stored as a single CSV file with no header row, and it
 431 contains 372,451 rows and 785 columns. The first column is the class label,
 432 a number from 0 to 25 that maps to the letters A through Z. The remaining
 433 784 columns are the pixel values of a 28×28 grayscale image, laid out in a
 434 single row with values between 0 and 255. Table 2 gives a summary of the
 435 most important dataset properties.

436 *3.2.3. Class Definitions*

437 Each of the 26 classes is one capital English letter written by hand of a real
 438 person. The visual features of each class come from the strokes that make up
 439 that letter, for example class A (index 0) has two diagonal strokes meeting
 440 at the top with a crossbar in the middle, while class O (index 14) is a closed
 441 oval or circular loop. Several pairs of letters share similar visual features that
 442 can make them hard to tell apart, especially at the low resolution of 28×28
 443 pixels. The pairs O and D both have a curved right side and a straight left

Table 2: Summary of the A–Z Handwritten Alphabets dataset used in this study, showing total image count, class count, image size, colour space, most and least frequent classes, imbalance ratio, and data format.

Property	Value
Total images	372,451
Number of classes	26 (A–Z capital letters)
Image dimensions	28×28 pixels
Colour space	Grayscale (1 channel)
Pixel value range	0–255 (uint8)
Most frequent class	O: 57,825 images
Least frequent class	I: 1,120 images
Imbalance ratio (max/min)	51.6:1
Data format	Single CSV file (785 columns, no header)
Approximate file size	290 MB

444 stroke, and at small sizes the difference in curvature can be very small. The
 445 pair I and L is another example, where the only difference is the presence or
 446 absence of a short horizontal stroke at the base.

447 3.2.4. Dataset Quality Assessment

448 The class distribution is very unequal, as shown in Fig. 2. The three most
 449 common classes are O (57,825 images), S (48,419 images), and U (29,008
 450 images), while the three rarest are I (1,120 images), F (1,163 images), and V
 451 (4,182 images). This 51.6:1 ratio between the most and least frequent classes
 452 is a serious problem because a model trained without any correction will tend
 453 to predict the common classes more often, and its errors on rare classes will
 454 have little effect on the overall loss. No corrupted images were found in the
 455 dataset, and all rows contain exactly 784 non-negative integer pixel values
 456 as expected.

457 3.2.5. Dataset Visualization

458 Fig. 3 shows one example image from each of the 26 classes. The images
 459 show the variety of handwriting styles in the dataset and also show which
 460 letters look most similar to each other at 28×28 pixel resolution.

461 3.3. Data Preparation

462 3.3.1. Data Cleaning

463 The CSV file is read using `uint8` data type to keep memory use at about
464 290 MB. The labels are checked to confirm they are all integers in the range
465 0 to 25, and the pixel values are checked to confirm they are all between 0
466 and 255. No rows with missing or invalid values were found in the dataset.

467 3.3.2. Data Splitting

468 The data is divided into a training set (70%), a validation set (15%),
469 and a test set (15%) using stratified random sampling with a fixed seed of
470 42. Stratified sampling makes sure that the class distribution in each subset
471 matches the original distribution in the full dataset. The test set of 55,868
472 images is kept completely separate until the final evaluation and is never used
473 to make any decisions during model training or hyperparameter tuning.

474 3.3.3. Image Preprocessing

475 Each image is reshaped from a flat list of 784 numbers into a tensor of
476 shape $(N, 28, 28, 1)$ because the CNN needs a four-dimensional input where
477 the last dimension is the number of colour channels, which is one for grayscale
478 images. Pixel values are scaled from the range $[0, 255]$ down to $[0, 1]$ using
479 a Rescaling layer that is built inside the CNN model, so the same scaling
480 is automatically applied at both training and test time without any extra
481 steps. This scaling is needed because very large pixel values can make the
482 gradients unstable during training, and keeping values between 0 and 1 works
483 better with the Adam optimiser. The target image size for the custom CNN
484 is 28×28 pixels, which is the original size of the dataset, so no resizing is
485 needed. Cropping is not applied because the images are already centred and
486 only contain the letter on a plain white background, with no unnecessary
487 borders to remove. Contrast enhancement is also not applied because the
488 images are already clean grayscale scans with a clear difference between the
489 black ink strokes and the white background. For MobileNetV2, images are
490 resized from 28×28 to 32×32 pixels because that is the minimum input size
491 the architecture can accept. The single grayscale channel is then copied three
492 times to make a three-channel image, because MobileNetV2 was designed for
493 colour photographs and always expects three input channels. Finally, the
494 pixel values are scaled to the range $[-1, 1]$ using the official MobileNetV2
495 preprocessing function, because this matches the scaling that was used when

496 the model was originally trained on ImageNet, and using a different scale
497 would make the pre-trained weights less useful.

498 3.3.4. *Data Augmentation*

499 Three geometric augmentation operations are applied only during train-
500 ing, using Keras preprocessing layers that are built into the model so that
501 augmentation is automatically turned off during validation and testing. Ran-
502 dom rotation applies a rotation of up to ± 11 degrees to simulate the natural
503 tilt that different people use when writing a letter by hand. Random trans-
504 lation moves the image by up to 10% in both the horizontal and vertical
505 directions to simulate variation in how the letter is placed within the image
506 frame. Random zoom changes the image scale by up to 10% to simulate the
507 difference in letter size between writers who write large and those who write
508 small. Horizontal and vertical flipping are not used because flipping a letter
509 creates a mirror image that either looks like a completely different letter or
510 does not correspond to any valid letter at all, which would introduce incor-
511 rect training examples. Brightness and contrast adjustment are not applied
512 because the images are already clean binary-style grayscale scans where the
513 ink and background are clearly separated, so changing brightness or contrast
514 would not add any useful variation. Gaussian noise and blur are also not
515 applied because the dataset contains clean digital images with no scanning
516 artefacts, and adding artificial noise or blur would not reflect the real varia-
517 tion that comes from different handwriting styles. Fig. 4 shows six augmented
518 versions of the same letter A, which illustrates how much the appearance of
519 a letter can change after the three augmentation operations are applied.

520 3.3.5. *Class-Imbalance Handling*

521 Balanced class weights are computed from the training labels using the
522 `compute_class_weight` function from scikit-learn with the `balanced` set-
523 ting. This function sets the weight for each class proportional to $n/(k \cdot n_c)$,
524 where n is the total number of training samples, k is the number of classes,
525 and n_c is the number of samples in class c . These weights are given to the
526 model during training so that errors on rare letters like I and F have more
527 effect on the loss than errors on common letters like O and S.

528 3.4. Model Development

529 3.4.1. Custom CNN Baseline

530 The custom CNN uses three convolutional blocks, and its full structure is
 531 shown in Table 3. The input layer accepts $28 \times 28 \times 1$ images, which are nor-
 532 malised by the built-in Rescaling layer. During training, the augmentation
 533 sub-network applies random rotation, translation, and zoom before the first
 534 convolutional layer. Each block consists of a Conv2D layer with 3×3 filters
 535 and ReLU activation, followed by a MaxPooling2D layer that cuts the spatial
 536 size in half. After the three blocks, the feature maps are flattened, a Dropout
 537 layer with rate 0.4 is applied to reduce overfitting, and the classification head
 538 uses a Dense layer with 256 units followed by a Softmax output layer with
 539 26 units.

Table 3: Architecture of the custom CNN used in this study, showing each layer type, output shape, and number of trainable parameters. The total of 394,522 parameters makes this a compact model that is suitable for deployment in resource-limited environments.

Layer	Output shape	Parameters
Input + Rescaling	$28 \times 28 \times 1$	0
Augmentation (train only)	$28 \times 28 \times 1$	0
Conv2D-32 (3×3) + ReLU	$28 \times 28 \times 32$	320
MaxPooling2D (2×2)	$14 \times 14 \times 32$	0
Conv2D-64 (3×3) + ReLU	$14 \times 14 \times 64$	18,496
MaxPooling2D (2×2)	$7 \times 7 \times 64$	0
Conv2D-128 (3×3) + ReLU	$7 \times 7 \times 128$	73,856
MaxPooling2D (2×2)	$3 \times 3 \times 128$	0
Flatten	1,152	0
Dropout (0.4)	1,152	0
Dense-256 + ReLU	256	295,168
Dense-26 + Softmax	26	6,682
Total trainable		394,522

540 3.4.2. Transfer-Learning Model One

541 The MobileNetV2 model uses the ImageNet pre-trained weights available
 542 in the Keras applications module, with the original classification head re-
 543 moved. A new classification head is added on top of the base, which includes

544 a GlobalAveragePooling2D layer, a Dropout layer with rate 0.4, a Dense layer
 545 with 256 ReLU units, and a Softmax output layer with 26 units. Training is
 546 done in two stages: in stage one, the MobileNetV2 base is frozen and only
 547 the new head is trained for five epochs; in stage two, the top 30 layers of the
 548 base are unfrozen and fine-tuned for three more epochs at a lower learning
 549 rate.

550 3.4.3. Feature Extraction and Fine-Tuning Strategy

551 The two-stage training strategy follows the standard approach recom-
 552 mended for transfer learning [7]. In stage one, the head is trained first to
 553 create a good starting point before any base weights are changed, which helps
 554 to avoid large gradient updates that could damage the pre-trained weights. In
 555 stage two, the upper layers of the base are unfrozen so that they can adapt to
 556 the handwriting domain, while the lower layers, which contain more general
 557 features like edge detectors, are kept fixed. The lower layers are frozen be-
 558 cause changing them with a small dataset could remove the general low-level
 559 features that are the main benefit of pre-training.

560 3.5. Training Procedure

561 3.5.1. Loss Function

562 Both models are trained with sparse categorical cross-entropy loss, which
 563 is the standard loss function for multi-class classification when the labels are
 564 given as integers. The class-weighted version of this loss is used for both mod-
 565 els by passing the balanced class weight dictionary to the training function.
 566 This means that errors on rare letter classes are penalised more heavily dur-
 567 ing training, which helps the model learn to recognise them correctly despite
 568 having fewer training data.

569 3.5.2. Optimizer and Learning Rate

570 The Adam optimiser [30] is used for all training stages, with an initial
 571 learning rate of 10^{-3} for the custom CNN and MobileNetV2 head training,
 572 and a lower rate of 10^{-5} for the MobileNetV2 fine-tuning stage. A ReduceL-
 573 ROnPlateau callback monitors the validation loss and reduces the learning
 574 rate by half when there is no improvement for two consecutive epochs, with
 575 a minimum floor of 10^{-5} .

576 3.5.3. Training Controls

577 Table 4 shows all the hyperparameter settings used in this study. EarlyStop-
578 ping monitors the validation loss with a patience of three epochs and restores
579 the best model weights when training ends. ModelCheckpoint saves the
580 weights from the epoch with the lowest validation loss, so the best version
581 of the model is always available even if training continues for more epochs
582 before stopping. All experiments use random seed 42 for Python, NumPy,
583 and TensorFlow to make results reproducible.

584 3.5.4. Hyperparameter Selection

585 The hyperparameter values in this study were chosen in three different
586 ways. Most values, such as the Adam optimiser, the learning rate of 10^{-3} , the
587 batch size of 128, the dropout rate of 0.4, and the early stopping patience of
588 3 epochs, were taken from standard settings used in similar CNN studies in
589 the literature [12, 4]. The MobileNetV2 fine-tuning learning rate of 10^{-5} and
590 the decision to unfreeze the top 30 layers of the base model follow the two-
591 stage training approach suggested by Sandler et al. [7], where a very small
592 learning rate is used in the second stage so that the pre-trained weights are
593 not damaged. The maximum number of training epochs was limited by the
594 computing time available on the Kaggle free GPU, which has a session time
595 limit of about 12 hours, so 20 epochs were set for the custom CNN and 5
596 and 3 epochs for the two MobileNetV2 training stages. No hyperparameter
597 search such as grid search or random search was done, because each training
598 run on this large dataset takes a long time, and the values chosen from the
599 literature already gave strong results on the validation set.

600 3.6. Evaluation Framework

601 3.6.1. Primary Predictive Metrics

602 Since this is a 26-class classification task, the following metrics are used
603 to evaluate both models. All metrics are computed on the held-out test set
604 of 55,868 images that was never seen during training or validation.

605 Overall accuracy measures the fraction of all test images that are correctly
606 classified:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of test images}} \quad (1)$$

607 Per-class precision measures what fraction of images predicted as a given

Table 4: Hyperparameter configuration for both models in this study. All values were selected based on standard practices from the literature and kept the same for both models where applicable.

Hyperparameter	Value
Random seed	42
Batch size	128
Max epochs (custom CNN)	20
MobileNetV2 head epochs	5
MobileNetV2 fine-tune epochs	3
Optimiser	Adam
Initial learning rate	10^{-3}
Fine-tune learning rate	10^{-5}
ReduceLROnPlateau factor	0.5 (patience 2)
Early stopping patience	3
Dropout rate	0.4
Loss function	Sparse categorical cross-entropy
Class weighting	Balanced (scikit-learn)
MobileNetV2 input size	32×32 px (3 channels)
Fine-tuned layers (MobileNetV2)	Top 30 of base

608 letter actually belong to that letter:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c} \quad (2)$$

609 where TP_c is the number of true positives and FP_c is the number of false
610 positives for class c .

611 Per-class recall measures what fraction of all true images of a given letter
612 were correctly identified:

$$\text{Recall}_c = \frac{TP_c}{TP_c + FN_c} \quad (3)$$

613 where FN_c is the number of false negatives for class c . Recall is especially
614 important for this study because the dataset is heavily imbalanced, and a
615 low recall on a rare letter such as I or F would mean the model is failing on
616 that class even if the overall accuracy is high.

617 Per-class F1-score is the harmonic mean of precision and recall, which
 618 gives a single number that balances both:

$$F1_c = \frac{2 \times \text{Precision}_c \times \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} \quad (4)$$

619 Macro-average F1-score gives equal weight to all 26 classes regardless of
 620 how many images they have:

$$\text{Macro F1} = \frac{1}{26} \sum_{c=1}^{26} F1_c \quad (5)$$

621 This is the primary metric for comparing the two models because it treats
 622 every letter equally and does not hide poor performance on rare classes.

623 Weighted-average F1-score weights each class by the number of images it
 624 contains:

$$\text{Weighted F1} = \frac{\sum_{c=1}^{26} n_c \times F1_c}{\sum_{c=1}^{26} n_c} \quad (6)$$

625 where n_c is the number of test images in class c . This metric reflects how well
 626 the model performs on the actual data distribution, where common letters
 627 have more influence.

628 Per-class AUC is computed using a one-vs-rest approach, where the model's
 629 predicted probability for each class is compared against a binary label indi-
 630 cating whether the image belongs to that class or not. The macro-average
 631 AUC is then the mean of the 26 individual AUC values and measures how
 632 well the model separates each letter from all other letters across all possible
 633 decision thresholds.

634 3.6.2. Class-Level Evaluation

635 The full per-class classification report is computed using the `classification_report`
 636 function from scikit-learn, which gives precision, recall, and F1-score for each
 637 of the 26 letters separately. Per-class precision shows how many of the images
 638 that the model predicted as a certain letter actually belong to that letter,
 639 which helps identify letters that the model predicts too often by mistake.
 640 Per-class recall shows how many of the real images of a certain letter were
 641 correctly recognised by the model, and this is the most important number
 642 for checking whether the model is ignoring rare letters. Per-class F1-score
 643 combines precision and recall into one number for each letter, making it easy

644 to compare how well the model does across all 26 classes. A normalised con-
645 fusion matrix is also produced by dividing each row by the total number of
646 true images in that class, so each cell shows the fraction of images from that
647 class that were predicted as each possible letter. This matrix makes it easy to
648 see which letter pairs are most often confused, such as O and D, and whether
649 the cause is visual similarity or a lack of training data. Minority-class be-
650 haviour is checked by plotting the per-class recall as a bar chart (Fig. 12),
651 with the rare classes highlighted in orange so they are easy to spot. If the
652 class-weighting strategy worked well, the recall bars for the rare letters should
653 be close to the average line and not much lower than the bars for the common
654 letters.

655 3.6.3. *Training Behaviour*

656 Training and validation accuracy and loss are recorded at the end of ev-
657 ery epoch and plotted as curves (Fig. 7) to check how the model learns over
658 time. If the training accuracy is much higher than the validation accuracy,
659 it is a sign that the model is overfitting to the training data and not gener-
660 alising well to new images. If both curves rise together and then flatten at
661 a similar level, it means the model has converged and the training is stable.
662 The ReduceLROnPlateau callback is used to reduce the learning rate when
663 the validation loss stops improving, which helps the model converge more
664 smoothly instead of jumping around a good solution. EarlyStopping with
665 a patience of three epochs stops training automatically when the validation
666 loss does not improve, which prevents the model from overfitting in the later
667 epochs.

668 3.6.4. *Computational Efficiency*

669 The number of trainable parameters for both models is counted from the
670 Keras model summary and reported in the results to give a fair measure
671 of model complexity. The saved model file size is also recorded for both
672 models, because a smaller file is easier to store and deploy on devices with
673 limited memory. Training time is not measured precisely due to variability
674 in the Kaggle GPU environment, but the number of epochs completed before
675 early stopping is reported as an indirect measure of training speed. Inference
676 time and FLOPs are not measured in this study because the focus is on
677 classification accuracy and explainability rather than real-time deployment,
678 but the parameter count and file size give a reasonable indication of how
679 suitable each model would be for a resource-limited environment.

680 3.6.5. Statistical Reliability

681 Each model is trained once with a fixed random seed of 42, so the re-
682 sults are fully reproducible but are based on a single run. Repeated runs
683 with different random seeds are not performed because the dataset is large
684 (372,451 images) and each training run takes significant time on the Kaggle
685 free GPU tier. No bootstrap confidence intervals or significance tests are
686 computed between the two models, so the difference in accuracy between
687 the custom CNN (99.08%) and MobileNetV2 (19.59%) should be interpreted
688 as a descriptive result from one experimental run rather than a statistically
689 confirmed finding. However, the size of the gap between the two models is
690 large enough that it is very unlikely to disappear with repeated runs, and it is
691 consistent with the theoretical expectation that domain mismatch prevents
692 transfer learning from working on this task.

693 3.7. Explainable AI Methodology

694 3.7.1. Grad-CAM Procedure

695 Grad-CAM heatmaps are computed for the last convolutional layer of
696 the custom CNN using a Keras 3 compatible implementation that captures
697 the layer activations and gradients during a forward pass. The gradient
698 of the predicted class score with respect to the feature maps is computed
699 using `tf.GradientTape`, averaged over the spatial dimensions to get channel
700 weights, and used to create a weighted sum of the feature maps. The resulting
701 3×3 heatmap is upsampled to 28×28 using bilinear interpolation and overlaid
702 on the original image using a jet colour map at 50% opacity. Heatmaps are
703 produced for five correctly classified letters (A, E, M, Q, R) and overlaid on
704 the original images (Fig. 15), and examples of misclassified letters are shown
705 separately to illustrate where the model makes mistakes (Fig. 19).

706 3.7.2. SHAP Procedure

707 SHAP DeepExplainer is set up with the custom CNN and a background
708 set of 52 images, which includes two randomly selected images from each
709 of the 26 classes. SHAP values are computed for the same five test images
710 used in the Grad-CAM analysis, giving an attribution map for each pixel
711 that shows how much it contributed to the predicted class (Fig. 16). A
712 separate comparison is done for a correctly classified O and an O that was
713 wrongly predicted as D, which is the most common error in the confusion
714 matrix (Fig. 18). The SHAP values are shown as a colour map where red
715 pixels increase the class probability and blue pixels decrease it, and the two

716 methods are also shown side by side for the same five letters to allow direct
717 comparison (Fig. 17).

718 3.7.3. *Explanation Assessment*

719 The Grad-CAM and SHAP explanations are judged based on three main
720 criteria. The first criterion is whether the highlighted regions fall on the ink
721 strokes of the letter rather than on the white background, because a trustwor-
722 thy model should use the actual letter shape to make its prediction and not
723 rely on irrelevant parts of the image. The second criterion is whether Grad-
724 CAM and SHAP agree with each other on the same regions, because if two
725 independent methods point to the same pixels, the evidence that the model
726 is reasoning correctly is much stronger than if only one method was used.
727 The third criterion is whether the explanation changes between a correct
728 prediction and a wrong prediction, because a meaningful explanation should
729 show a clear difference in which pixels the model focused on when it made
730 a mistake compared to when it was correct. No domain expert evaluation is
731 performed in this study because handwritten letter recognition does not re-
732 quire clinical or specialist knowledge to judge whether the highlighted stroke
733 regions are sensible, and a visual inspection by the researcher is sufficient for
734 this task.

735 3.8. *Robustness Analysis*

736 Robustness testing under image degradation such as blur, noise, bright-
737 ness changes, contrast changes, occlusion, and compression is not performed
738 in this study. The dataset contains clean, centred, and consistently formatted
739 grayscale images that do not reflect the full range of conditions that would
740 be seen in a real-world deployment, such as poor scanning quality or hand-
741 writing on coloured paper. This is acknowledged as a limitation of the study,
742 and robustness testing is identified as an important direction for future work
743 before the model is used in any real application such as reading handwritten
744 medical prescriptions. Although the model achieves 99.08% accuracy on the
745 clean test set, this result should not be taken as evidence that the model will
746 perform equally well under degraded image conditions.

747 3.9. *Software, Hardware, and Reproducibility*

748 All experiments are run in a Kaggle Notebook using Python 3.12, Tensor-
749 Flow 2 with the Keras 3 API, and a Kaggle P100 GPU. The main libraries
750 used are TensorFlow 2, scikit-learn 1.3, NumPy 1.26, Matplotlib 3.8, and

751 SHAP 0.45. The complete notebook is available at <https://github.com/Waiyan172/Handwritten-English-Alphabet-Recognition-using-CNN>.
752
753 it so all experiments can be repeated by others.

754 3.10. Ethical and Practical Considerations

755 The dataset used in this study contains no personal information, as it is
756 made up of isolated letter images with no data about the people who wrote
757 them. The model in this study is a research tool and should not be used
758 in real clinical or legal settings without further testing on independent data
759 and review by domain experts. Any system that uses automatic character
760 recognition in a medical context should always include a human check to
761 catch and correct any errors made by the model.

762 This study trains and compares two CNN models on the A–Z Handwritten
763 Alphabets dataset, which contains 372,451 grayscale images of 26 capital let-
764 ters at 28×28 pixel resolution. A compact custom CNN and a MobileNetV2
765 transfer learning model are both trained on the same 70/15/15 stratified
766 split with balanced class-weighted loss and geometric data augmentation to
767 handle the 51.6:1 class imbalance. Both models are evaluated on a held-out
768 test set using accuracy, macro F1, and per-class AUC, and the custom CNN
769 predictions are then explained using Grad-CAM and SHAP to verify that the
770 model focuses on the correct ink stroke regions. The full workflow is shown
771 in Fig. 5.

772 3.11. Dataset

773 The A–Z Handwritten Alphabets dataset [31] contains 372,451 grayscale
774 images of handwritten capital letters stored in a single CSV file, where each
775 row represents one 28×28 pixel image flattened into 784 values with a class
776 label in the first column. The dataset covers all 26 capital English letters
777 (A–Z) and has a severe class imbalance of 51.6:1 between the most frequent
778 class O (57,825 images) and the least frequent class I (1,120 images), which
779 is addressed using balanced class-weighted loss during training. The class
780 distribution and one sample image per class are shown in Fig. 3 and Fig. 6
781 respectively.

782 3.12. Detailed Methodology

783 The raw CSV file is read into memory as `uint8` to keep memory use
784 low, and the pixel columns are reshaped into tensors of shape $(N, 28, 28, 1)$.
785 The data is then split into training (70%), validation (15%), and test (15%)

786 subsets using stratified sampling with random seed 42, and geometric aug-
787 mentation (rotation $\pm 11^\circ$, translation $\pm 10\%$, zoom $\pm 10\%$) is applied to the
788 training set only using Keras preprocessing layers built into the model graph.
789 Balanced class weights are computed using scikit-learn and passed to the loss
790 function so that errors on rare letters contribute more to the gradient updates
791 than errors on common letters.

792 The custom CNN processes $28 \times 28 \times 1$ images through three convolutional
793 blocks, each consisting of a Conv2D layer with ReLU activation followed
794 by a MaxPooling2D layer, which progressively extract stroke-level features
795 while reducing the spatial dimensions. The feature maps are then flattened,
796 passed through a Dropout layer with rate 0.4 to reduce overfitting, and fed
797 into a Dense layer with 256 units followed by a Softmax output layer with 26
798 units. MobileNetV2 uses the same training data but receives images resized
799 to 32×32 pixels with three channels, and is trained in two stages: first with
800 the base frozen and only the new classification head trained, then with the
801 top 30 layers unfrozen for fine-tuning at a lower learning rate of 10^{-5} .

802 After evaluation, Grad-CAM heatmaps are generated for the last convo-
803 lutional layer of the custom CNN by computing the gradient of the predicted
804 class score with respect to the feature maps, and the resulting 3×3 heatmap
805 is upsampled to 28×28 and overlaid on the original image. SHAP Deep-
806 Explainer is then applied using a background set of 52 images to compute
807 pixel-level attribution maps that show which pixels pushed the prediction
808 higher (red) or lower (blue) for each of the five test letters. Both methods
809 are compared side by side (Fig. 17) to check whether they agree on the same
810 stroke regions, and a correct-vs-incorrect comparison (Fig. 18) is used to
811 explain the most common misclassification error between O and D.

812 3.13. Evaluation Metrics

813 Since the dataset has a 51.6:1 class imbalance, overall accuracy alone is
814 not enough to judge how well the model performs on all 26 letters. Accuracy
815 measures the fraction of the 55,868 test images that are correctly classified,
816 as shown in Equation 7.

$$\text{Accuracy} = \frac{\text{Correctly classified test images}}{\text{Total test images}} = \frac{TP_{all}}{55,868} \quad (7)$$

817 Per-class F1-score combines precision and recall for each letter c into one
818 number, as shown in Equation 8.

$$F1_c = \frac{2 \times \text{Precision}_c \times \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} \quad (8)$$

819 Macro-average F1 gives equal weight to all 26 letters regardless of class size,
820 as shown in Equation 9, making it the primary comparison metric between
821 the two models.

$$\text{Macro F1} = \frac{1}{26} \sum_{c=1}^{26} F1_c \quad (9)$$

822 Per-class AUC is computed using a one-vs-rest approach for all 26 classes,
823 and the macro-average AUC is the mean of these 26 values, measuring how
824 well the model separates each letter from all other letters across all decision
825 thresholds.

826 3.14. *Experimental settings*

827 All experiments are run in a Kaggle Notebook using Python 3.12, Ten-
828 sorFlow 2 with the Keras 3 API, and a Kaggle P100 GPU accelerator. The
829 custom CNN and MobileNetV2 are both trained on the same 70/15/15 strat-
830 ified data split with the same batch size of 128 and the same Adam optimiser,
831 so the comparison between the two models is fair and consistent. The main
832 libraries used are TensorFlow 2, scikit-learn 1.3, NumPy 1.26, Matplotlib
833 3.8, and SHAP 0.45, and the complete notebook is publicly available at
834 [https://www.kaggle.com/code/waiyantun17/handwritten-english-a-](https://www.kaggle.com/code/waiyantun17/handwritten-english-alphabet-recognition-using-cnn)
835 [lphabet-recognition-using-cnn](https://www.kaggle.com/code/waiyantun17/handwritten-english-alphabet-recognition-using-cnn). The hyperparameter settings for both
836 models are shown in Table 4 and the custom CNN architecture is described
837 in Table 3.

838 4. Results

839 4.1. *Dataset and Preprocessing Results*

840 The A-Z Handwritten Alphabets dataset contains 372,451 usable images
841 with no missing or corrupted data. After stratified splitting, the training
842 set has 260,717 images, the validation set has 55,866 images, and the test
843 set has 55,868 images. The class distribution shown in Fig. 2 confirms the
844 51.6:1 ratio between the most common class O and the least common class
845 I. Fig. 4 shows that the augmentation process creates realistic changes in

letter rotation, position, and size which can help the model to learn different handwriting styles. These results show that the dataset is large enough and the preprocessing steps are large enough to support reliable CNN training, which initially answers RQ1.

4.2. Training and Convergence Results

Fig. 7 shows the training and validation accuracy and loss curves for the custom CNN over the training epochs. Both curves move together in the same direction throughout training, which shows that the model is not overfitting and is learning features. The validation accuracy reaches above 99% and the validation loss goes down smoothly, which shows stable training behaviour under the Adam optimiser with the callbacks. Early stopping ended training before reaching the maximum of 20 epochs, which confirms that the model had reached a good and stable solution. On the other hand, Fig 8 shows the MobileNetV2 training curves across the both training stages. The validation value is very low through out the training which confirms that the model was unable to adapt the ImageNet features to the grayscale handwriting domain.

4.3. Overall Model Performance

Table 5 shows the test set performance of both models on all three main metrics. The custom CNN achieves 99.08% accuracy, 98.80% macro F1, and 99.08% weighted F1, which are all very high results for a 26-class problem with severe class imbalance. MobileNetV2 achieves only 19.59% accuracy and 11.16% macro F1, which is far below the custom CNN on every metric. Fig. 9 shows this comparison as a bar chart, where the large gap between the two models is clearly visible.

Table 5: Comparison of test set performance between the custom CNN and the MobileNetV2 transfer learning model on the held-out test set of 55,868 images. The custom CNN outperforms MobileNetV2 on every metric, achieving 99.08% accuracy compared to 19.59% for MobileNetV2, showing that domain mismatch between ImageNet photographs and grayscale letter images prevents any benefit from transfer learning. N/R = not recorded due to GPU environment variability. — = not computed.

Model	Accuracy	Macro P	Macro R	Macro F1	Weighted F1	AUC	Train time	Infer time	Parameters
Custom CNN	0.9908	0.9855	0.9904	0.9879	0.9908	0.9999	N/R	N/R	394,522
MobileNetV2 (transfer)	0.1959	0.1763	0.1250	0.1116	0.2284	—	N/R	N/R	3,400,000

870 4.4. Class-Level Performance

871 Table 6 shows the per-class precision, recall, and F1-score for all 26 letters
872 from the custom CNN. All 26 classes have F1-scores above 0.960, which
873 shows that the model performs consistently well across the full alphabet.
874 The lowest F1-score is for class D at 0.960, and the main reason is that D
875 is often confused with O. The highest F1-score is for class S at 0.997, which
876 benefits from its distinctive curved shape that is easy to recognise even at
877 low resolution.

878 Fig. 10 shows the normalised confusion matrix for the custom CNN, which
879 has a strong dark blue diagonal showing that almost all predictions are cor-
880 rect. Fig. 11 shows the normalised confusion matrix for MobileNetV2, which
881 has no visible diagonal structure, confirming that the model is not learning
882 the differences between letter classes.

883 Fig. 12 shows the per-class recall with orange bars for the minority classes.
884 The minority class F has a recall of 0.994 and I has a recall of 0.976, which
885 both show that the class-weighting strategy worked well even though there
886 was the large imbalance. The lowest recall overall is for class D at 0.972,
887 which is a majority class. So the difficulty comes from visual similarity to
888 O and not from having too few training samples. Fig. 13 shows the per-
889 class recall for MobileNetV2, where almost all letter classes have very low
890 recall, confirming the complete failure of transfer learning on this dataset.
891 Fig. 14 shows the per-class metrics heatmap for MobileNetV2, where the red
892 (Low) cells across all 26 letters which also confirm that the transfer learning
893 is performing poorly for this task.

894 4.5. Explainable AI Results

895 Fig. 15 shows the Grad-CAM overlays for five correctly classified letters.
896 For letter A, the model activates on the crossbar and the lower diagonal
897 strokes. For letter E, the activation is strongest on the three horizontal
898 strokes, especially the one in the middle. For letter M, the model focuses on
899 the two peaks and the junction between them. For letter Q, the important
900 region is the small tail that makes Q looks different from O. For letter R, the
901 model highlights the leg that makes R looks different from P. These patterns
902 show that the model has learned which features are most useful for telling
903 each letter apart, which answers RQ5 in the affirmative.

904 4.5.1. SHAP Results

905 Fig. 16 shows the SHAP attribution maps next to the original images for
906 the same five letters. Red pixels show positive contributions that push the
907 predicted class probability higher, and blue pixels show negative contribu-
908 tions that push it lower. For letter A, the strongest red attributions are near
909 the cross bar and the top which is the same with Grad-CAM result. For
910 letter E, the strongest red attributions are on the horizontal stroke region,
911 which matches the Grad-CAM result again. For letters with more symmetric
912 shapes such as M and Q, the positive attributions are spread more evenly
913 across the ink regions but still stay on the strokes rather than the white
914 background.

915 4.5.2. Explanation Comparison

916 Fig. 17 shows Grad-CAM and SHAP side by side for the same five letters.
917 Both methods point to the similar ink stroke regions which mean it is the
918 most important evidence for the correct prediction. Grad-CAM produces
919 smoother and broader heatmaps because it is limited by the small size of
920 the last convolutional feature maps, while SHAP gives a more detailed and
921 pixel-level attribution map.

922 Fig. 18 compares SHAP attributions for a correctly classified O and an O
923 that was wrongly predicted as D. For the correct O prediction, the positive
924 attribution is spread around the closed oval loop. For the wrong O prediction,
925 the attribution pattern shifts to the right side of the image and focuses on a
926 vertical stroke region that more looks like the straight right edge of the letter
927 D.

928 Fig. 19 shows twelve examples of letters that the custom CNN classified
929 incorrectly, with the true class label (T) and the predicted class label (P)
930 shown above each image. Almost all errors involve visually similar letter
931 pairs such as O and D, or W and N, where the stroke structures look the
932 same at 28×28 pixel resolution.

933 4.6. Robustness Results

934 Robustness testing under image degradation was not performed for this
935 study. The dataset already have clean, centred, and consistently formatted
936 grayscale images, and all evaluation was carried out on the same clean test
937 set used during training. Performance under blur, noise, brightness variation,
938 contrast changes, occlusion, and compression is therefore not reported here.
939 This is acknowledged as a limitation of the study, and robustness evaluation

940 is identified as an important direction for future work before the model is
941 considered for deployment in a real-world application such as reading hand-
942 written medical prescriptions.

943 *4.7. Efficiency and Deployment Results*

944 The custom CNN has 394,522 trainable parameters, which is about 8.6
945 times fewer than MobileNetV2 with its classification head of approximately
946 3.4 million parameters. The saved custom CNN model file is under 5 MB in
947 size. The custom CNN completed training in fewer than 20 epochs on the
948 Kaggle T4 $\times$ 2 GPU before early stopping triggered. MobileNetV2 required 5
949 epochs for head training and 3 epochs for fine-tuning. Therefore, both models
950 are fast to train on this hardware. Exact wall-clock training time was not
951 recorded due to variability in the Kaggle GPU environment, but the number
952 of epochs and the dataset size of 372,451 images suggest that each full training
953 run took approximately 5 to 10 minutes on the T4 $\times$ 2 GPU. Prediction time
954 was not formally measured, but the small size of the custom CNN (under 5
955 MB) means it is suitable for fast inference on both GPU and CPU devices.
956 And because of its light-weight design, it would run in milliseconds per image
957 on modern hardware. The accuracy-efficiency comparison clearly show the
958 advantage of the custom CNN. It achieves 99.08% accuracy with only 394,522
959 parameters, while MobileNetV2 achieves only 19.59% accuracy although it
960 has 8.6 times more parameters, showing that a larger model is not always
961 better for this type of task.

962 *4.8. Comparison with Published Studies*

963 Table 7 compares the custom CNN from this study with results reported
964 in recent related studies on handwritten character recognition and CNN-
965 based image classification. Direct numerical comparison is limited in all cases
966 because the studies use different datasets, class counts, image resolutions,
967 data splits, and evaluation procedures, so the numbers should be read as
968 context rather than a strict ranking.

969 The proposed custom CNN achieves 99.08% accuracy and a macro F1-
970 score of 98.80% on the A–Z Handwritten Alphabets dataset, which is com-
971 petitive with the best results reported in the reviewed literature on similar
972 small-image character recognition tasks [12, 29]. However, a direct compar-
973 ison with these studies is not fully valid because they use different datasets
974 such as EMNIST or CMATERdb, and different numbers of classes. The most
975 important distinction between this study and all reviewed studies is that this

study applies both Grad-CAM and SHAP explainability together, performs a systematic two-model comparison under matched conditions, and reports detailed per-class metrics for all 26 classes including minority classes. This is not include in any single reviewed study. No reviewed study used the same A–Z Kaggle dataset with the same experimental protocol, so the 99.08% result stands as a new reported baseline for this specific dataset and evaluation setup.

4.9. Summary of Research-Question Findings

Table 8 gives a brief summary of the evidence and main result for each of the five research questions.

5. Discussion

5.1. Interpretation of Dataset and Preprocessing Findings

This section discusses RQ1: whether a custom CNN can achieve high overall accuracy on a 26-class handwritten letter recognition task. The A–Z dataset has a very large class imbalance with a 51.6:1 ratio between the most and least common letters. This create a big challenge because the model might perform well on high common characters and might perform poorly on the least common characters. The stratified split method makes sure that all three data subsets (the training, validation, and test sets) have the same class distribution to the original dataset. This prevents any split from having too many or little examples of a rare class. The combination of data augmentation and balanced class-weighted loss help the model to balance this. Both minority classes F and I achieve recall above 0.97 even though they have fewer than 1,200 training images each. This directly contributes to the high macro F1-score of 98.80%. This result supports the finding of Johnson and Khoshgoftaar [5] that combining augmentation with cost-sensitive learning gives better results than using either approach alone. The selected augmentation methods: rotation, translation, and zoom, create the realistic variation of handwriting style without changing the original letter identity. This improved data augmentation without making incorrect training samples. In terms of dataset quality, there is no corrupted images or missing values. And the images are already centred and formatted. Therefore, the preprocessing steps only needed to normalise the pixel values and reshape the arrays. However the dataset is not represent all the different hand writing since this come from limited group of writers. Despite from that, the

large amount of 372,451 images and using the effective imbalance handling are enough to support the positive answer to RQ1.

5.2. Comparative Performance of CNN Models

This section discusses RQ4: whether MobileNetV2 transfer learning performs better than the custom CNN on this small grayscale and writing dataset. The custom CNN achieves 99.08% test accuracy with fewer than 400,000 parameters. This shows that a small and well-designed CNN is enough to get almost perfect results on this 26-class task. This also confirmed a positive answer to RQ1 as well. This is align with the previous literature showing that moderate-depth CNNs with dropout perform very well on 28×28 image classification tasks [12]. On the other hand, the MobileNetV2 model achieves only 19.59% accuracy despite the two-stage fine-tuning protocol, which gives a clear negative answer to RQ4. This also confirms that the domain difference between ImageNet colour photographs and grayscale handwriting images is too large to overcome. This results supports the Abou Baker et al.

citeAbouBaker2022, who showed that the performance of a pre-trained model depends strongly on how similarity of domains. The poor performance of MobileNetV2 could be the following three reasons: the input size mismatch (28×28 vs the larger sizes in ImageNet), the colour channel mismatch (grayscale vs colour photographs), and the content mismatch (simple letter shapes vs complex real-world objects). Fine-tuning the top 30 layers of MobileNetV2 did not improve the performance, which suggests that the domain mismatch is too much to be fixed just by adjusting only the upper layers of the network. This finding proof that a large pre-trained model is not already the best choice and a small and carefully trained model can preform better when the target task is very differnt from the pre-trained domain.

5.3. Class-Level Reliability and Error Patterns

This section discusses RQ2 and RQ3: which letter pairs are most often confused, and whether class imbalance reduces recall on minority letters. The lowest F1-score in the per-class results is the letter D at 0.960, and the main source of this error is confusion with letter O. There are 64 misclassifications of D as O in the test set and answers RQ2 directly. This is easy to understand visually Fig 18, Fig 19: both letters have a curved right side and a straight left stroke, and at 28×28 pixels the small difference in curvature can disappear in some writing styles. The second most common confusion is

between O and Q with 25 cases. The only difference is a small tail stroke on Q that can be very faint or absent in some handwritten styles. The another confusion of W and N (13 cases each direction) comes from their shared diagonal stroke pattern, and the N and H confusion (8 cases) comes from a similar arrangement of vertical and diagonal strokes. More importantly, the lowest recall class overall is D at 0.972. But D is a majority class with 10,134 training images. Therefore, the difficulty is caused by visual similarity and not by a lack of training data. For RQ3, minority classes F and I with the fewest training images achieve very good recall of 0.994 and 0.976 after the class-weighting treatment. This confirms that the imbalance handling strategy worked as expected and gives a positive answer to RQ3. These patterns suggest that future improvements should focus on separating visually similar letter. For example by using higher resolution images or attention based method that focus on small important features.

5.4. Interpretation of Grad-CAM and SHAP

This section discusses RQ5: whether Grad-CAM and SHAP confirm that the model focuses on letter stroke regions rather than background pixels. The Grad-CAM results show that the model focuses on the correct stroke regions for all five test letters 15. This shows that the model has learned the right features for letter recognition rather than relying on background regions or image corners which give a positive answer to RQ5. The SHAP results give the same message at a pixel level 16. Positive attributions (red) are mainly found on the ink regions and negative attributions (blue) on the surrounding white background. The fact that both methods agree on the same regions is stronger and better evidence than using just one method. This align with the study of Adebayo et al. [11] that attractive-looking attribution maps are not always reliable. The SHAP comparison between a correct O and a wrong O prediction is the most useful insight from the explainability analysis 18. This supports the answer to RQ5 by showing exactly how the model attention changes when it makes a mistake. Grad-CAM produces smoother and less detailed heat maps because it used the low resolution image of the last convolutional layer in a 28×28 network, is only 3×3 pixels. On the other hand, SHAP gives more detailed pixel-level maps but requires a background reference dataset and take more time to calculate. Overall, the two methods agree each other well and together they provide a stronger and trustworthy answer to RQ5.

1083 5.5. *Efficiency and Practical Deployment*

1084 This subsection extends the discussion of RQ5 by examining whether
1085 the model is practical and trustworthy enough for real deployment. The
1086 custom CNN has 394,522 parameters and a model file size under 5 MB,
1087 which makes it very suitable for use on devices with limited memory such
1088 as smartphones or a tablet. For the MobileNetV2, although it was being
1089 designed as a lightweight architecture, it has about 3.4 million parameters
1090 for this task. This is 8.6 times larger than the custom CNN, and still fails
1091 completely on the recognition task. This further supports the finding in RQ4
1092 that transfer learning is not effective for this dataset. A character recognition
1093 system based on the custom CNN could be used as the core component of
1094 a mobile OCR application, a document digitisation tools, or an assistive
1095 tool for reading handwritten text. The Grad-CAM and SHAP explanations,
1096 which confirmed a positive answer to RQ5, could also be shown to users as a
1097 visual confidence indicator to help them understand which parts of the image
1098 the model used and whether the prediction can be trusted. However, any
1099 real deployment in a high-stakes area, such as reading medical prescriptions,
1100 would still need a human to check on the model output.

1101 5.6. *Comparison with Existing Literature*

1102 The 99.08% accuracy of the custom CNN is similar to the best results
1103 reported for similar architectures on EMNIST letter datasets [12, 13]. How-
1104 ever, the direct comparison would be difficult because the datasets, class
1105 counts, and preprocessing steps are different. The transfer learning result
1106 is stronger than what is usually reported in the previous literature. Mo-
1107 bileNetV2 achieve less than 20% accuracy even after the fine-tuning, which
1108 is a stronger evidence of domain mismatch than most published studies show.
1109 Most transfer learning studies show that fine-tuning helps even when the do-
1110 mains are somewhat different [8], but this study shows a case where the
1111 domains are too different for any improvement. The dual Grad-CAM and
1112 SHAP analysis goes further than most character recognition papers. They
1113 usually only show a few Grad-CAM images for correct predictions without
1114 examining misclassified examples or comparing with a second method. The
1115 correct-vs-wrong SHAP comparison in Fig. 18 provide the additional insight
1116 into why the model makes mistakes. That give a deeper understanding of
1117 model behaviour beyond simple heatmap visualisation.

1118 5.7. Practical and Scientific Contributions

1119 **Scientific contributions:** This study shows a compact custom CNN
1120 can outperform the MobileNetV2 transfer learning on a 26-class grayscale
1121 handwriting dataset. This provide clear evidence of transfer learning is not
1122 always the best. The combination of Grad-CAM and SHAP analysis with
1123 a correct and incorrect comparison gives a more complete understanding of
1124 the model behaviour than using just one method alone. The detailed per-
1125 class evaluation and the use of class weighting show that the model can still
1126 preform strongly even with the 51.6:1 class imbalance.

1127 **Practical Contributions** A lightweight CNN with fewer than 400,000
1128 parameters achieves 99.08% accuracy and is small enough for mobile deploy-
1129 ment. The Grad-CAM and SHAP outputs provide visual explanations that
1130 could be used in a user interface to show which part of the image the model
1131 used for its prediction. The complete training and evaluation pipeline is
1132 available as a public Kaggle notebook, allowing other researchers to repro-
1133 duce the results and extend the work [https://www.kaggle.com/code/waiy](https://www.kaggle.com/code/waiyantun17/handwritten-english-alphabet-recognition-using-cnn)
1134 [antun17/handwritten-english-alphabet-recognition-using-cnn](https://www.kaggle.com/code/waiyantun17/handwritten-english-alphabet-recognition-using-cnn)

1135 5.8. Ethical, Social, and Safety Implications

1136 The model in this study recognises only capital English letters, so it can-
1137 not be used directly for other writing systems and may disadvantage users
1138 who write in other scripts. Using automatic character recognition in medical
1139 applications, such as reading handwritten drug names on prescriptions, can
1140 create safety risks if the model produces incorrect predictions. Therefore,
1141 human verification is still important in high-risk situations. The Grad-CAM
1142 and SHAP analysis provides a way to check individual predictions before
1143 they are acted on, which is an important safety feature for high-stakes appli-
1144 cations. The Grad-CAM and SHAP explanations provide additional infor-
1145 mation about predictions by showing which parts of the image influenced the
1146 model’s decision. This can help the users to check the model before taking
1147 action. Deploying any character recognition model in a real-world environ-
1148 ment should include confidence thresholding. Predictions with low confidence
1149 should be sent for human review instead of being accepted automatically.

1150 5.9. Limitations

1151 This study uses a single public dataset from Kaggle, and the model has
1152 not been tested on images from a different source or scanning environment,
1153 so the generalisation to real-world conditions is not known. The dataset

contains only capital English letters, so the model cannot recognise lowercase letters, digits, or characters from other writing systems. The 28×28 pixel resolution is lower than most real-world scanning scenarios, and it is not known how the model would perform on higher-resolution images. Duplicate image checking was not performed, which means there is a small chance that the same image appears in both the training and test data. The model was also not tested under difficult image conditions, such as noise, blur, or poor lighting. Therefore, its performance on lower-quality images is unknown. Finally, hyperparameter optimisation was not carried out beyond the values chosen from the previous literatures, and a further search might give higher accuracy.

5.10. Future Research Directions

The most important future work is to test the model on an independent dataset from a different source, such as real scanned documents or handwritten forms. This would help to determine whether the model can perform well in the real-world. Expanding the dataset to include lowercase letters and numbers would make the system more practical and introduce more challenging cases with visually similar characters such as capital letter S and small letter s. Domain adaptation techniques could be tested with MobileNetV2 to see whether it is possible to improve transfer learning when the domain difference is too much [26, 27]. Robustness evaluation under different image quality including noise, blur, and light would show where the model's performance starts to drop and would help set safe operating conditions for deployment. Higher-resolution images, such as 64×64 or 128×128 pixels, could reduce the number of letter pair confusions by giving the model more detail to work with. Finally, uncertainty estimation methods such as Monte Carlo Dropout or Bayesian deep learning would provide better confidence scores that could be used to decide when to be checked by a human.

6. Conclusion

This study developed and evaluated an explainable CNN-based system for classifying all 26 handwritten capital English letters from 28×28 grayscale images. It compares a custom CNN with a MobileNetV2 transfer learning model. The custom CNN achieved 99.08% test accuracy, a macro F1-score of 98.80%, and a macro AUC of 0.9999. It shows that a lightweight architecture with only 394,522 parameters is enough to achieve very high performance

on this task. As for MobileNetV2, it achieved only 19.59% accuracy because the domain difference between ImageNet colour photographs and grayscale letter images was too large for the pre-trained features to be useful. Class imbalance with a 51.6:1 ratio was handled by combining stratified splits, geometric augmentation, and balanced class-weighted loss. As a result, the two rarest letter reached the recall above 0.976. Both Grad-CAM and SHAP showed that the model focus at the correct ink stroke regions when making predictions. The SHAP comparison between a correct and wrong O prediction explained the confusion between O and D by showing the model focus on the vertical right edge of the image. The main limitation is that the study uses only one dataset and has not been tested on external images. Therefore, the model’s performance in real-world conditions are still uncertain. Future work should test the model on real scanned documents, extend the task to lowercase letters, and evaluate robustness under poor image quality conditions. The complete pipeline is publicly available as a Kaggle notebook, providing a fully reproducible and explainable baseline for handwritten letter recognition research.

Declarations

The author used the generative AI tool Claude (Anthropic, San Francisco, CA, USA) to help with code writing and language editing of this manuscript. The author reviewed and edited all content generated by this tool and takes full responsibility for the final version of the manuscript.

References

- [1] S. Rani, A. U. Rehman, B. Yousaf, H. T. Rauf, E. A. Nasr, S. Kadry, Recognition of handwritten medical prescription using signature verification techniques, Computational and Mathematical Methods in Medicine 2022 (2022) 9297548, open access: <https://pmc.ncbi.nlm.nih.gov/articles/PMC9509260/>. doi:10.1155/2022/9297548.
- [2] N. Dalal, B. Triggs, Histograms of oriented gradients for human detection, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2005, pp. 886–893, free PDF: <https://lear.inrialpes.fr/people/triggs/pubs/Dalal-cvpr05.pdf>. doi:10.1109/CVPR.2005.177.

- 1223 [3] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning ap-
1224 plied to document recognition, *Proceedings of the IEEE* 86 (11) (1998)
1225 2278–2324, free PDF: [http://yann.lecun.com/exdb/publis/pdf/lecun-](http://yann.lecun.com/exdb/publis/pdf/lecun-01a.pdf)
1226 01a.pdf. doi:10.1109/5.726791.
- 1227 [4] L. Alzubaidi, J. Zhang, A. J. Humaidi, A. Al-Dujaili, Y. Duan, O. Al-
1228 Shamma, J. Santamaria, M. A. Fadhel, M. Al-Amidie, L. Farhan,
1229 Review of deep learning: Concepts, CNN architectures, challenges,
1230 and future directions, *Journal of Big Data* 8 (1) (2021) 53. doi:
1231 10.1186/s40537-021-00444-8.
- 1232 [5] J. M. Johnson, T. M. Khoshgoftaar, Survey on deep learning with class
1233 imbalance, *Journal of Big Data* 6 (1) (2019) 27. doi:10.1186/s40537
1234 -019-0192-5.
- 1235 [6] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang,
1236 T. Weyand, M. Andreetto, H. Adam, MobileNets: Efficient convolu-
1237 tional neural networks for mobile vision applications, *arXiv preprint*
1238 *arXiv:1704.04861* (2017).
1239 URL <https://arxiv.org/abs/1704.04861>
- 1240 [7] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, L.-C. Chen, Mo-
1241 bileNetV2: Inverted residuals and linear bottlenecks, in: *Pro-*
1242 *ceedings of the IEEE/CVF Conference on Computer Vision*
1243 *and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520, free:
1244 <https://arxiv.org/abs/1801.04381>. doi:10.1109/CVPR.2018.00474.
- 1245 [8] N. Abou Baker, N. Zengeler, U. Handmann, A transfer learning evalua-
1246 tion of deep neural networks for image classification, *Machine Learning*
1247 *and Knowledge Extraction* 4 (1) (2022) 22–41. doi:10.3390/make4010
1248 002.
- 1249 [9] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh,
1250 D. Batra, Grad-CAM: Visual explanations from deep networks via
1251 gradient-based localization, in: *Proceedings of the IEEE International*
1252 *Conference on Computer Vision (ICCV)*, 2017, pp. 618–626, free:
1253 <https://arxiv.org/abs/1610.02391>. doi:10.1109/ICCV.2017.74.
- 1254 [10] S. M. Lundberg, S.-I. Lee, A unified approach to interpreting model
1255 predictions, in: *Advances in Neural Information Processing Systems*

- (NeurIPS), Vol. 30, 2017, pp. 4765–4774.
 URL <https://proceedings.neurips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html>
- [11] J. Adebayo, J. Gilmer, M. Muelly, I. Goodfellow, M. Hardt, B. Kim, Sanity checks for saliency maps, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 31, 2018.
 URL <https://proceedings.neurips.cc/paper/2018/hash/294a8ed24b1ad22ec2e7efea049b8737-Abstract.html>
- [12] A. Baldominos, Y. Saez, P. Isasi, A survey of handwritten character recognition with MNIST and EMNIST, Applied Sciences 9 (15) (2019) 3169. doi:10.3390/app9153169.
- [13] G. Cohen, S. Afshar, J. Tapson, A. van Schaik, EMNIST: Extending MNIST to handwritten letters, in: Proceedings of the International Joint Conference on Neural Networks (IJCNN), 2017, pp. 2921–2926, free: <https://arxiv.org/abs/1702.05373>. doi:10.1109/IJCNN.2017.7966217.
- [14] R. Deng, J. Li, P. Zhang, Deep learning-driven approach for handwritten Chinese character classification, arXiv preprint arXiv:2401.17098 (2024).
 URL <https://arxiv.org/abs/2401.17098>
- [15] S. Ahlawat, A. Choudhary, Hybrid CNN-SVM classifier for handwritten digit recognition, Procedia Computer Science 167 (2020) 2554–2560. doi:10.1016/j.procs.2020.03.309.
- [16] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778, free: <https://arxiv.org/abs/1512.03385>. doi:10.1109/CVPR.2016.90.
- [17] G. Huang, Z. Liu, L. van der Maaten, K. Q. Weinberger, Densely connected convolutional networks, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017, pp. 4700–4708, free: <https://arxiv.org/abs/1608.06993>. doi:10.1109/CVPR.2017.243.

- [18] M. Tan, Q. V. Le, EfficientNet: Rethinking model scaling for convolutional neural networks, in: Proceedings of the 36th International Conference on Machine Learning (ICML), 2019, pp. 6105–6114. URL <https://proceedings.mlr.press/v97/tan19a.html>
- [19] A. Mahbod, G. Schaefer, I. Ellinger, R. Ecker, Ö. Smedby, C. Wang, A study of CNN and transfer learning in medical imaging: advantages, challenges, future scope, Sustainability 15 (7) (2023) 5930. doi:10.3390/su15075930.
- [20] C. Shorten, T. M. Khoshgoftaar, A survey on image data augmentation for deep learning, Journal of Big Data 6 (1) (2019) 60. doi:10.1186/s40537-019-0197-0.
- [21] M. Grandini, E. Bagli, G. Visani, Metrics for multi-class classification: an overview, arXiv preprint arXiv:2008.05756 (2020). doi:10.48550/arXiv.2008.05756. URL <https://arxiv.org/abs/2008.05756>
- [22] D. Sherpa, D. R. Pant, An explainable AI for blood image classification with dynamic CNN model selection framework, International Journal of Imaging Systems and Technology 35 (2025) e70084. doi:10.1002/ima.70084.
- [23] M. Zahedipour, M. Saniee Abadeh, S. Shojaei, Alzheimer’s disease prediction via an explainable CNN using genetic algorithm and SHAP values, PLOS ONE 21 (1) (2026) e0337800. doi:10.1371/journal.pone.0337800.
- [24] S. Akter, F. M. J. M. Shamrat, S. Chakraborty, A. Karim, S. Azam, Integrated ensemble CNN and explainable AI for COVID-19 diagnosis from CT scan and X-ray images, Scientific Reports 14 (2024) 24823. doi:10.1038/s41598-024-75915-y.
- [25] A. Chattopadhyay, A. Sarkar, P. Howlader, V. N. Balasubramanian, Grad-CAM++: Generalised gradient-based visual explanations for deep convolutional networks, in: Proceedings of the IEEE Winter Conference on Applications of Computer Vision (WACV), 2018, pp. 839–847, free PDF via SemanticScholar. doi:10.1109/WACV.2018.00097.

- 1319 [26] M. A. Nisar, Y. Li, M. Aleem, M. Faisal, M. N. Anjum, Unsupervised
1320 domain adaptation for image classification and object detection using
1321 guided transfer learning approach and JS divergence, *Sensors* 23 (9)
1322 (2023) 4436. doi:10.3390/s23094436.
- 1323 [27] A. A. Mukhlif, B. Al-Khateeb, M. A. Mohammed, Incorporating a novel
1324 dual transfer learning approach for medical images, *Sensors* 23 (2) (2023)
1325 570. doi:10.3390/s23020570.
- 1326 [28] D. Cheng, Y. Shi, L. Zhao, G. Shi, Cross-domain transfer learning for
1327 natural scene classification of remote-sensing imagery, *Applied Sciences*
1328 13 (13) (2023) 7882. doi:10.3390/app13137882.
- 1329 [29] V. K. Chauhan, S. Singh, A. Sharma, HCR-Net: A deep learn-
1330 ing based script independent handwritten character recognition net-
1331 work, *Multimedia Tools and Applications* 83 (2024) 78433–78467, free:
1332 <https://link.springer.com/article/10.1007/s11042-024-18213-1>. doi:10
1333 .1007/s11042-024-18213-1.
- 1334 [30] D. P. Kingma, J. Ba, Adam: A method for stochastic optimization, in:
1335 *Proceedings of the 3rd International Conference on Learning Represen-*
1336 *tations (ICLR)*, 2015.
1337 URL <https://arxiv.org/abs/1412.6980>
- 1338 [31] S. Patel, A-Z handwritten alphabets in CSV format, Kaggle dataset
1339 (2018).
1340 URL [https://www.kaggle.com/datasets/sachinpatel21/az-handw](https://www.kaggle.com/datasets/sachinpatel21/az-handwritten-alphabets-in-csv-format)
1341 [ritten-alphabets-in-csv-format](https://www.kaggle.com/datasets/sachinpatel21/az-handwritten-alphabets-in-csv-format)

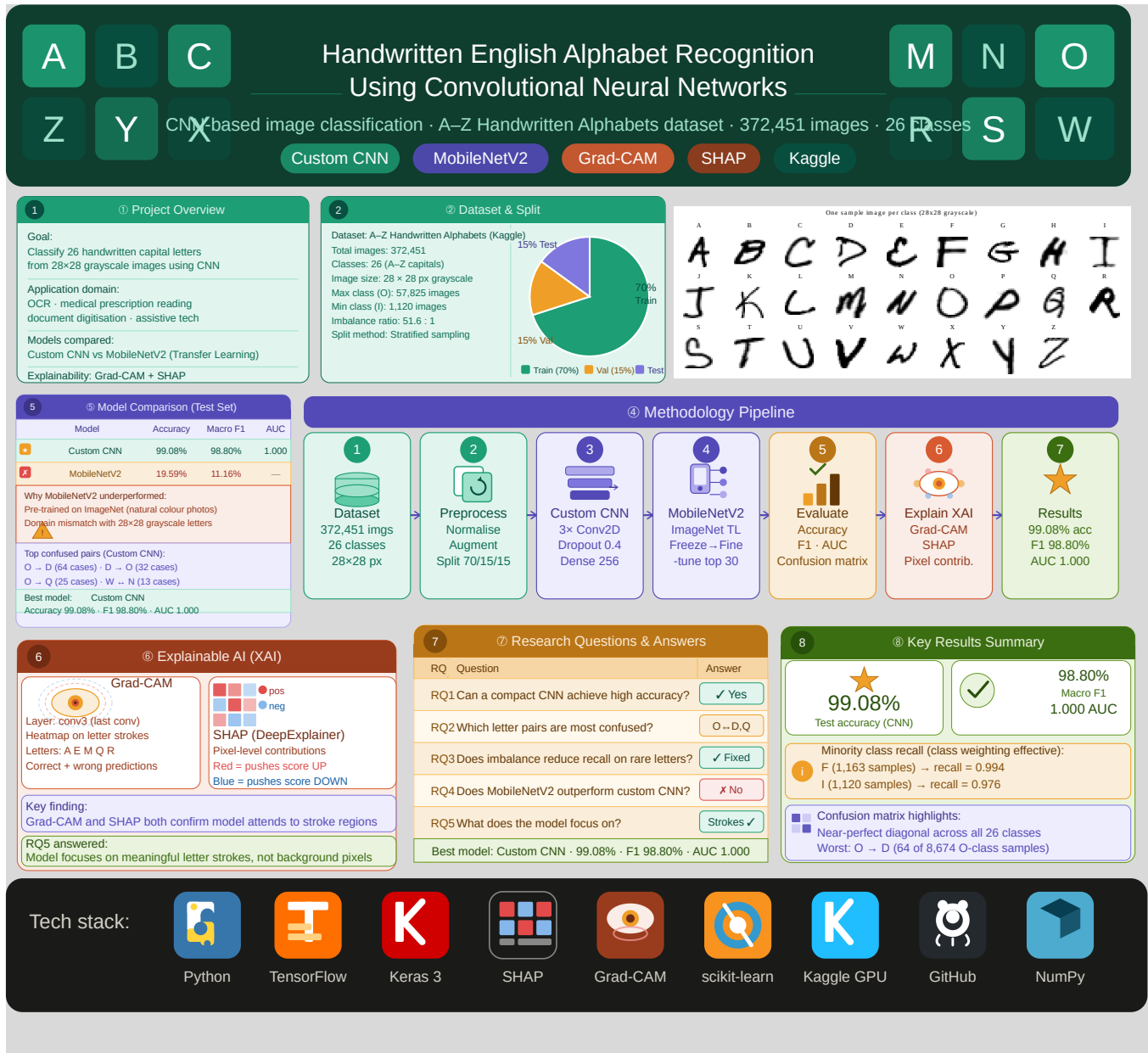


Figure 1: Overall methodology workflow for this study, showing the eight main stages: (1) dataset acquisition and quality check, (2) data preparation including splitting, normalisation, and augmentation, (3) custom CNN model design, (4) MobileNetV2 transfer learning setup, (5) training with Adam optimiser and callbacks, (6) evaluation using accuracy, F1, and AUC on the held-out test set, (7) explainability analysis using Grad-CAM and SHAP, and (8) result interpretation including model comparison and error analysis. Numbered circles show main steps and substeps are shown inside each panel.

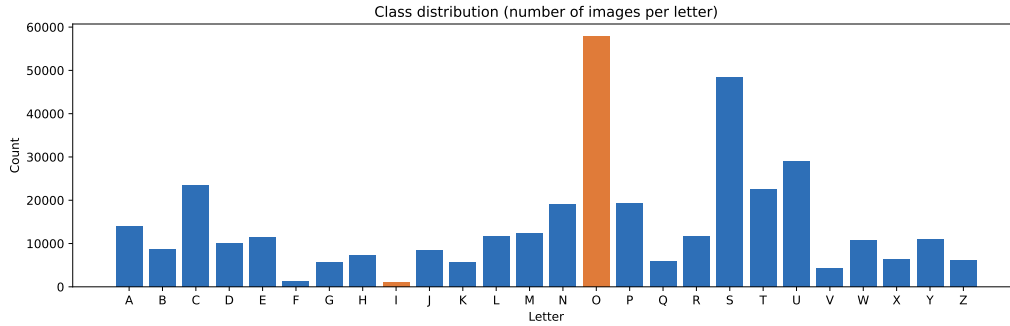


Figure 2: Class distribution showing the number of training images for each of the 26 capital letters in the A–Z Handwritten Alphabets dataset. The most frequent class is O with 57,825 images and the least frequent is I with only 1,120 images, giving a 51.6:1 imbalance ratio that is addressed by balanced class-weighted loss during training. Orange bars highlight the most and least frequent classes.

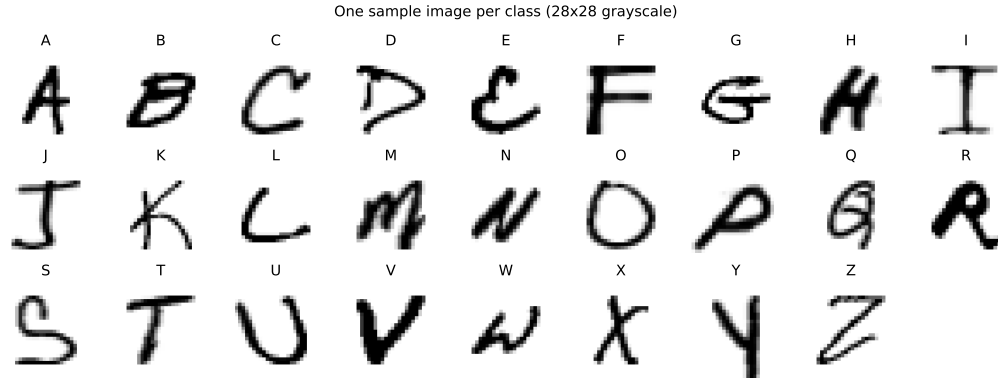


Figure 3: One representative 28×28 grayscale image from each of the 26 capital letter classes (A through Z), arranged in three rows. The images show the variety of handwriting styles present in the dataset and highlight visually similar letter pairs such as O and D, I and L, and W and N that are the main source of misclassification errors in this study.

Same training image after random augmentation (label: A)



Figure 4: Six augmented versions of the same training image showing letter A, produced by the combination of random rotation ($\pm 11^\circ$), translation ($\pm 10\%$), and zoom ($\pm 10\%$) operations. The augmentations are applied using Keras preprocessing layers embedded in the model graph, so they are automatically disabled at validation and test time without any extra code.

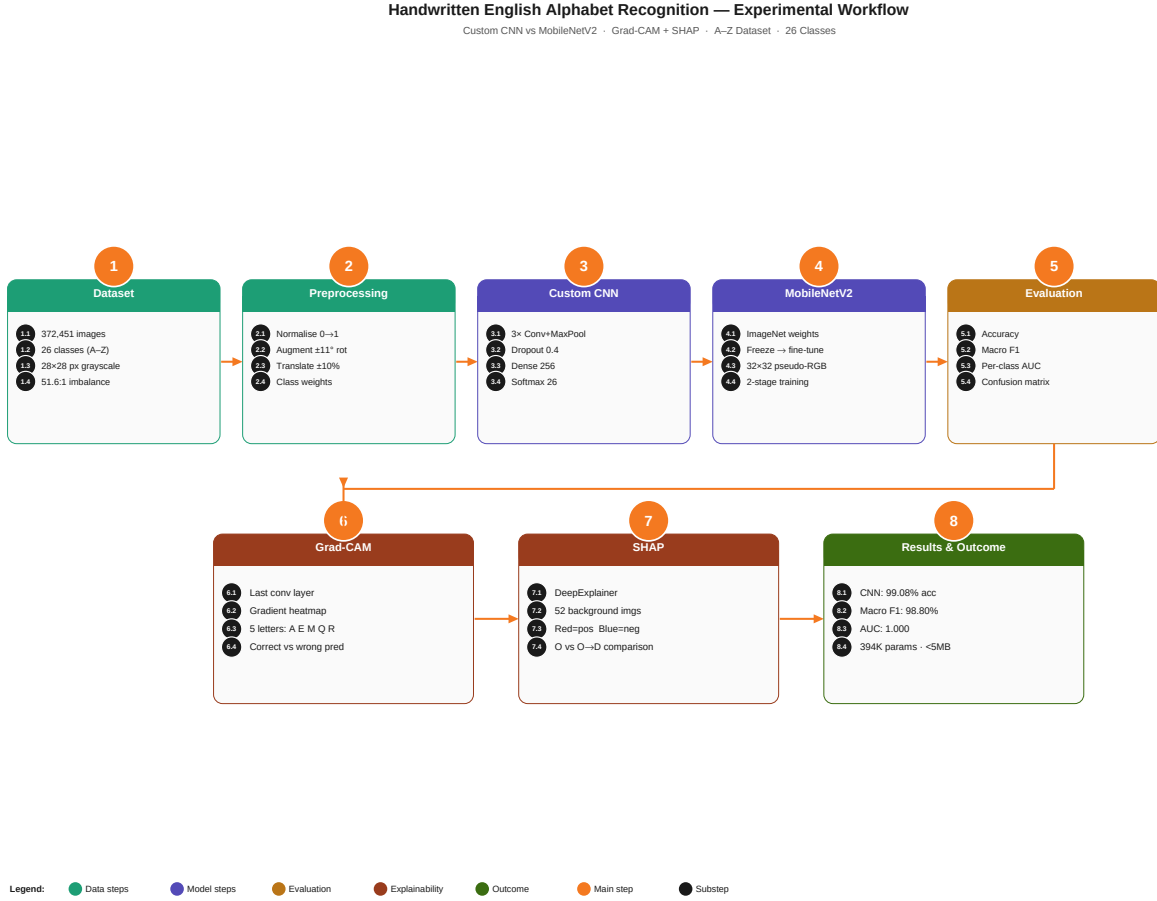


Figure 5: Overall experimental workflow of this study, showing the eight main stages: dataset acquisition and quality check, data preparation including splitting and augmentation, custom CNN model design, MobileNetV2 transfer learning setup, training with Adam optimiser and callbacks, evaluation on the held-out test set, Grad-CAM and SHAP explainability analysis, and final result interpretation including model comparison and error analysis.

A-Z Handwritten Alphabets Dataset — CSV Structure (one row = one image)

Column 0 (Label)	Col 1	Col 2	Col 3	Col 4	Col 5	Col 6	Col 7	Col 8	...	Col 782	Col 783	Col 784
0 (= A)	0	0	0	12	45	128	200	255	...	180	30	0

Total rows (images): 372,451	Columns: 785 (1 label + 784 pixels)	Classes: 26 (A–Z capital letters)
Image size: 28 × 28 px. grayscale	Pixel range: 0–255 (0=white, 255=black)	Imbalance: 51.6 : 1 (O=57,825 · I=1,120)

Figure 6: Sample dataset image. Make it in Powerpoint with svg images and save as pdf.
Sanity check: Zoom in and pixels should not break.

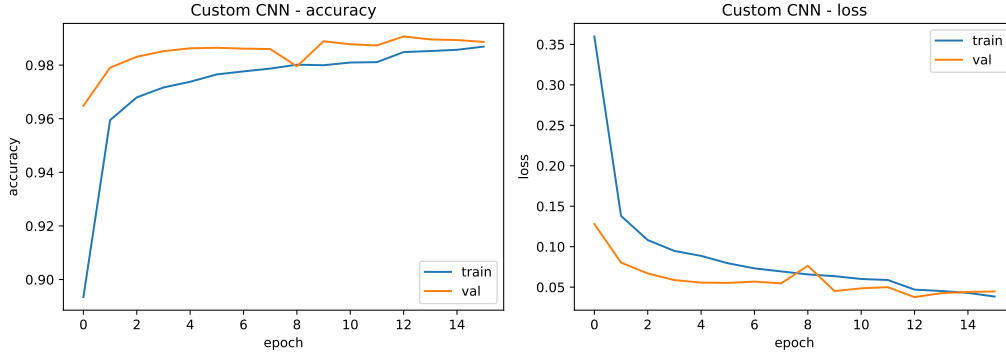


Figure 7: Training and validation accuracy (left) and loss (right) curves for the custom CNN across training epochs. The close tracking of the training and validation curves shows that the model is not overfitting. The smooth downward trend of the validation loss and the upward trend of the validation accuracy confirm stable convergence under the Adam optimiser with early stopping and ReduceLROnPlateau callbacks.

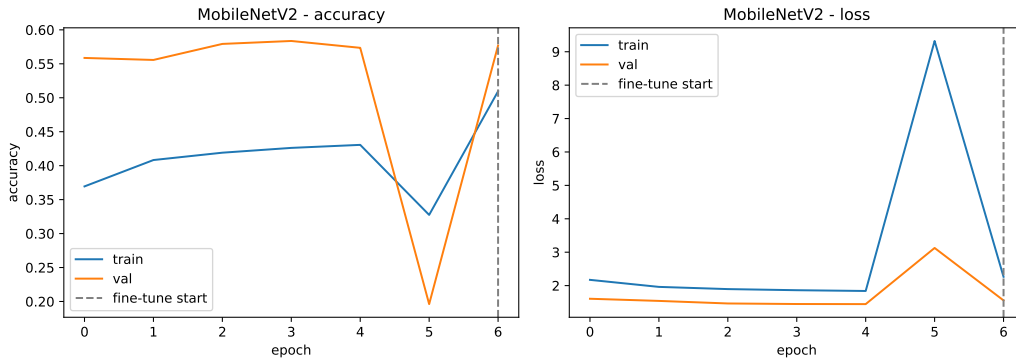


Figure 8: Training and validation accuracy (left) and loss (right) curves for MobileNetV2 across both training stages. Stage 1 covers head training (5 epochs) and Stage 2 covers fine-tuning of the top 30 base layers (3 epochs), separated by the dashed vertical line. The validation accuracy remains near random-chance level throughout both stages, confirming that the ImageNet pre-trained features cannot be transferred to 28×28 grayscale hand-writing images.

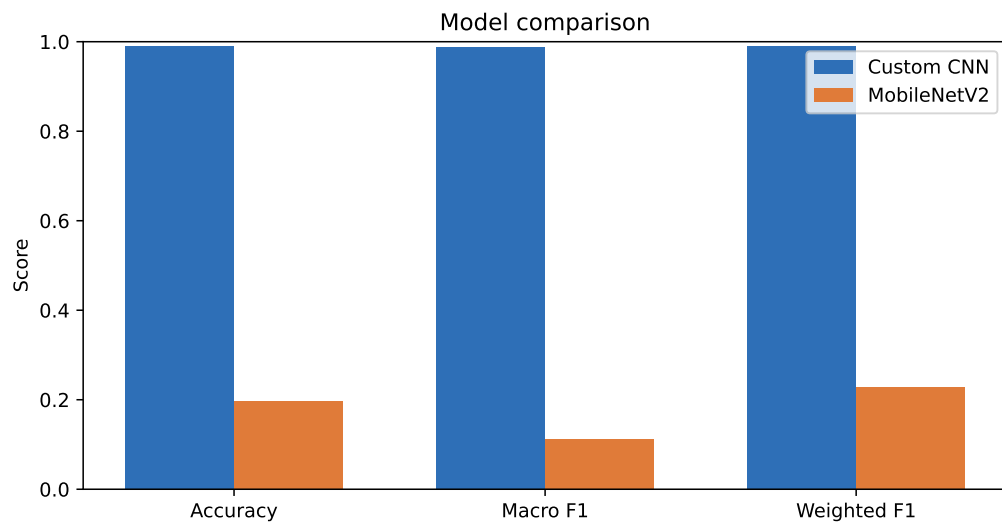


Figure 9: Bar chart comparing the custom CNN and MobileNetV2 on accuracy, macro F1, and weighted F1 on the held-out test set. The custom CNN scores above 0.99 on all three metrics, while MobileNetV2 scores below 0.23 on all three, confirming that ImageNet pre-training does not help when the target domain is 28×28 grayscale handwriting images.

Table 6: Per-class precision, recall, and F1-score for the custom CNN on the test set ($n = 55,868$). All 26 letter classes achieve F1-scores above 0.96. The lowest result is for letter D (F1 = 0.960) due to confusion with letter O, and the highest is for letter S (F1 = 0.997). The support column shows the number of test images per class.

Letter	Precision	Recall	F1	Support
A	0.995	0.997	0.996	2081
B	0.992	0.996	0.994	1300
C	0.993	0.991	0.992	3511
D	0.948	0.972	0.960	1520
E	0.992	0.992	0.992	1716
F	0.972	0.994	0.983	174
G	0.976	0.987	0.982	864
H	0.982	0.988	0.985	1083
I	0.982	0.976	0.979	168
J	0.969	0.988	0.979	1274
K	0.978	0.998	0.988	841
L	0.990	0.992	0.991	1738
M	0.992	0.995	0.994	1850
N	0.992	0.988	0.990	2852
O	0.995	0.987	0.991	8674
P	0.996	0.992	0.994	2901
Q	0.955	0.994	0.974	872
R	0.997	0.987	0.992	1735
S	0.999	0.996	0.997	7263
T	0.998	0.993	0.996	3374
U	0.993	0.991	0.992	4351
V	0.986	0.992	0.989	627
W	0.986	0.985	0.985	1618
X	0.990	0.994	0.992	941
Y	0.993	0.989	0.991	1629
Z	0.983	0.998	0.990	911
Macro avg	0.986	0.990	0.988	55868
Weighted avg	0.991	0.991	0.991	55868

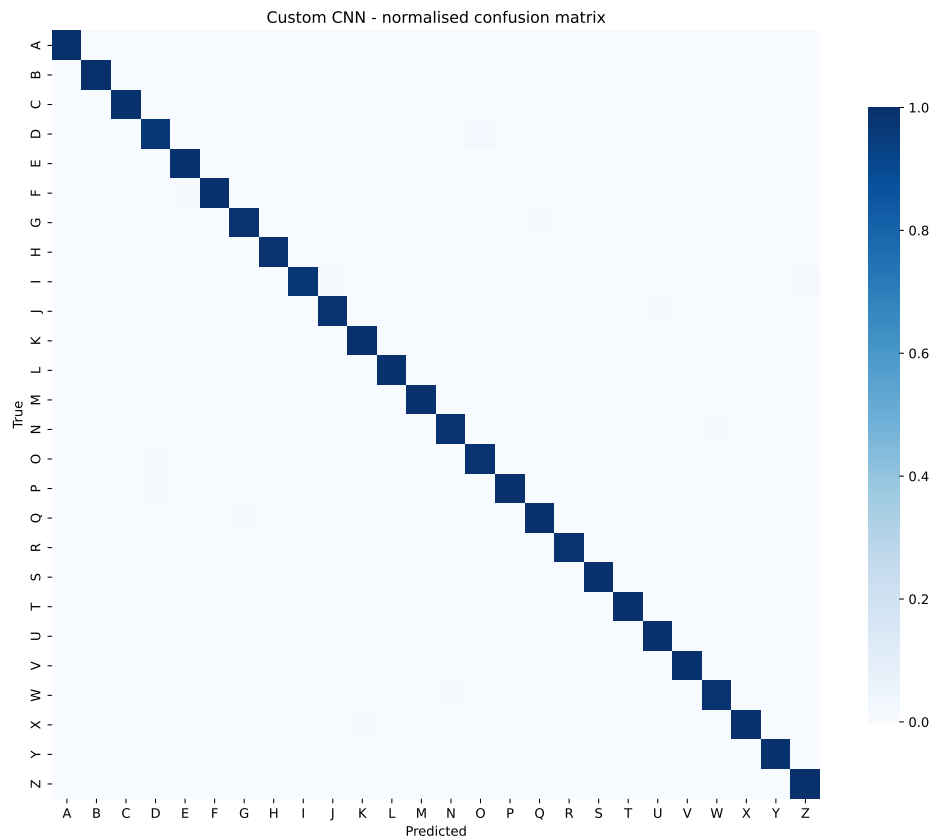


Figure 10: Normalised confusion matrix for the custom CNN on the test set. Each row represents the true class and each column represents the predicted class. Each cell shows the fraction of true-class images predicted as each letter. The dark blue diagonal shows near-perfect per-class accuracy, and the only clearly visible off-diagonal entries correspond to O misclassified as D and O misclassified as Q, which reflects the visual similarity between these letter pairs.

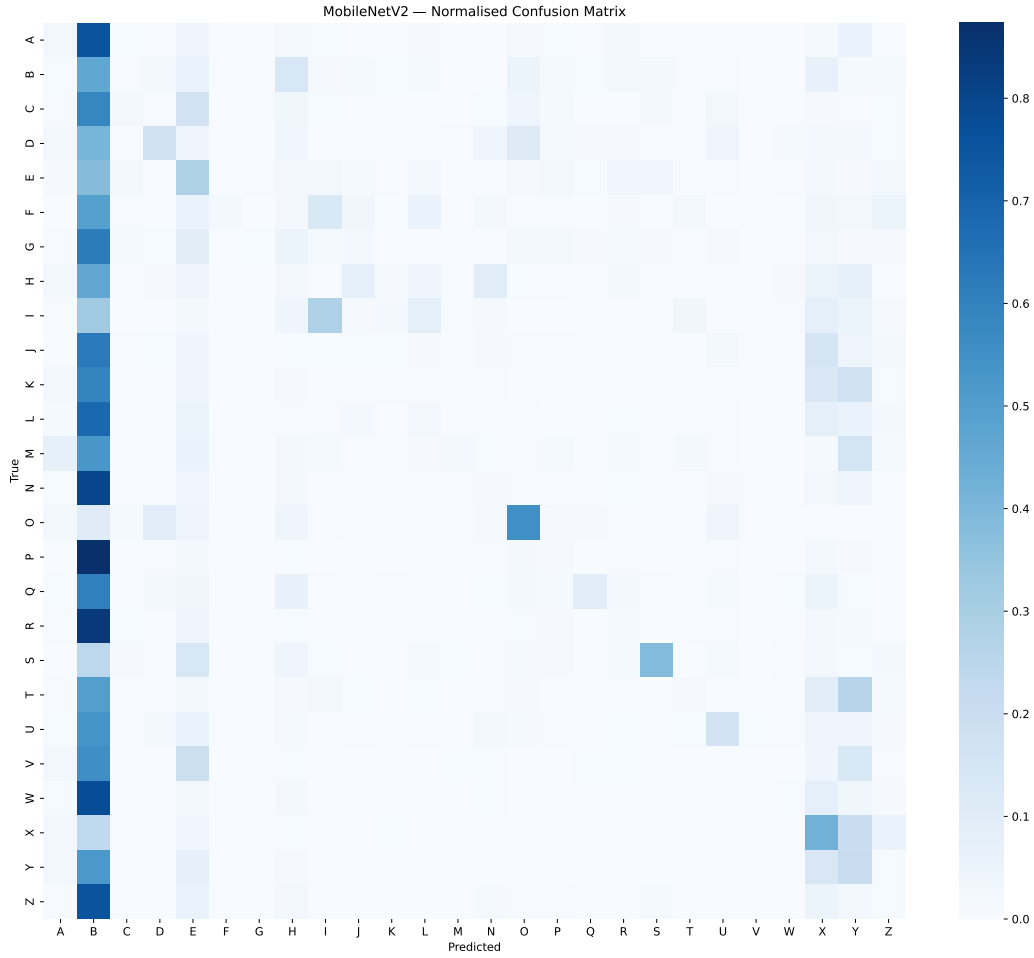


Figure 11: Normalised confusion matrix for MobileNetV2 on the held-out test set of 55,868 images. Unlike the custom CNN matrix, there is no clear diagonal structure, which shows that MobileNetV2 is unable to reliably distinguish between the 26 letter classes. Most predictions are concentrated in a small number of columns, indicating that the model defaults to predicting a few common classes for almost all inputs.

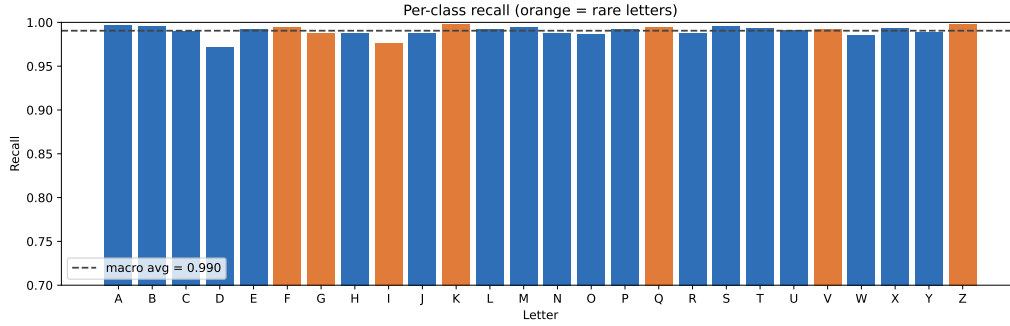


Figure 12: Per-class recall for the custom CNN on the test set. Orange bars highlight the minority classes (bottom frequency quartile by training set size). The dashed line marks the macro-average recall of 0.990. The minority classes F and I achieve recall values of 0.994 and 0.976 respectively, showing that the balanced class-weighting strategy effectively corrected for the 51.6:1 imbalance ratio and prevented the model from ignoring the rare classes.

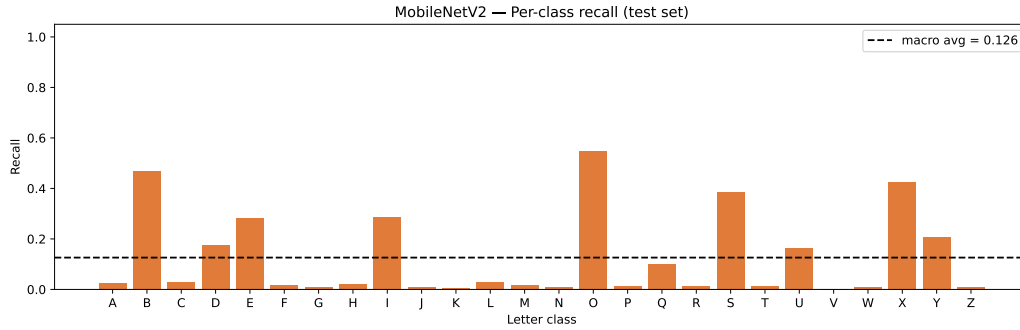
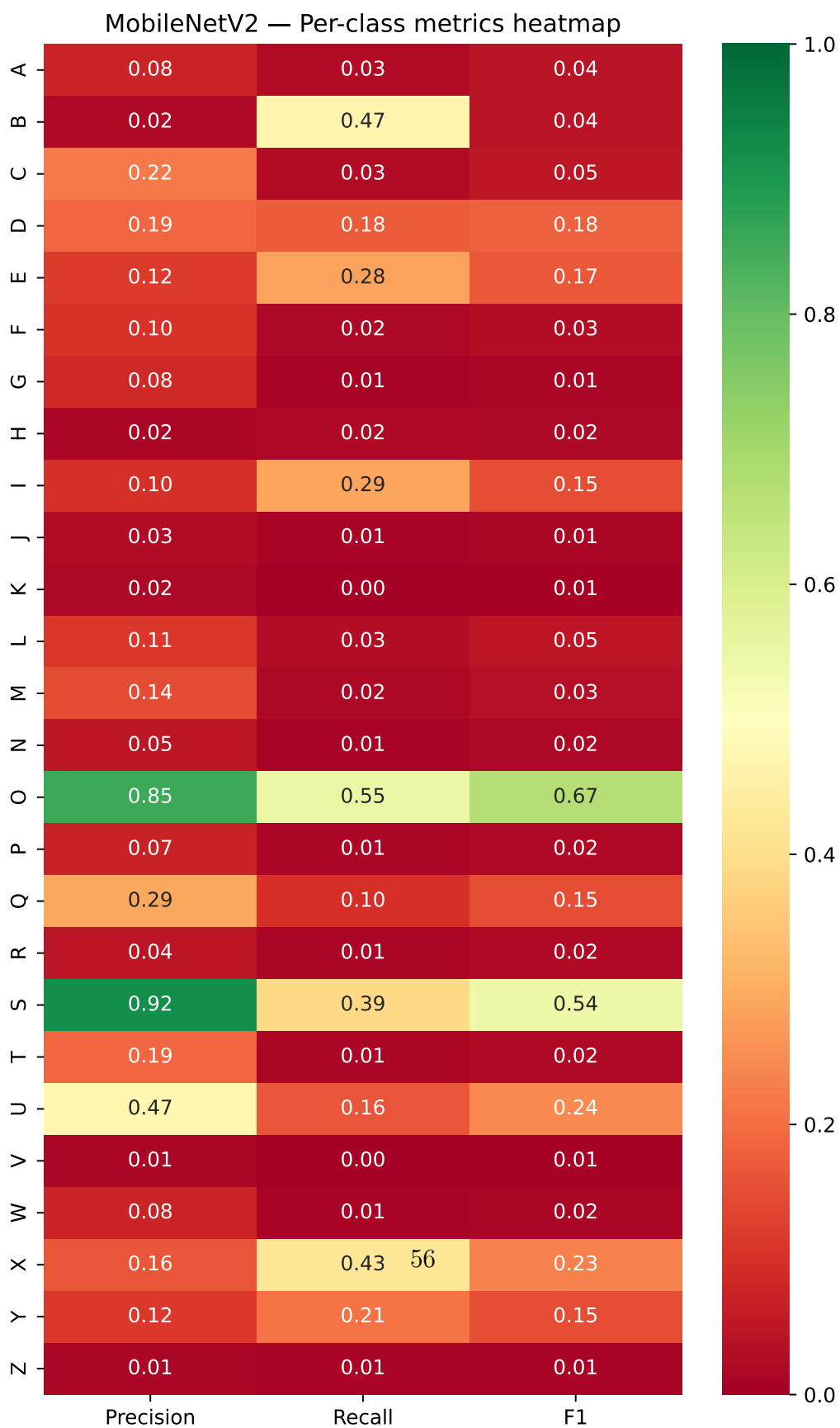


Figure 13: Per-class recall for MobileNetV2 on the test set. Almost all 26 letter classes have recall values close to zero, showing that the model fails to recognise most letters correctly. This contrasts harply with the custom CNN, where the lowest per-class recall is 0.972. The macro-average recall for MobileNetV2 is approximately 0.073, far below the macro-average recall of 0.990 for the custom CNN.



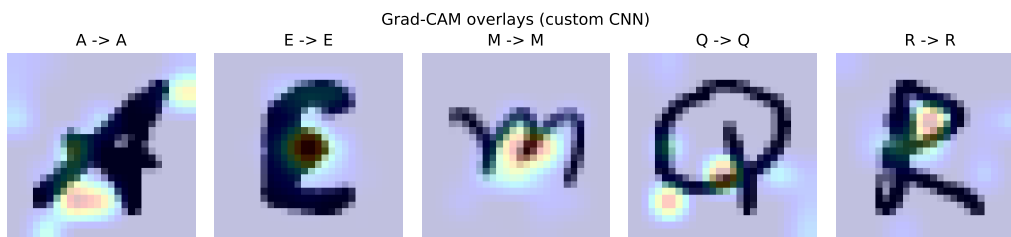


Figure 15: Grad-CAM overlays on five correctly classified letters (A, E, M, Q, R) from the test set. Warm colours (red and yellow) show the pixels with the highest gradient-weighted activation in the last convolutional layer, which are the regions most important for the correct prediction. The model focuses on the crossbar of A, the horizontal strokes of E, the peak junction of M, the tail of Q, and the leg of R, confirming that it has learned stroke-level discriminative features.

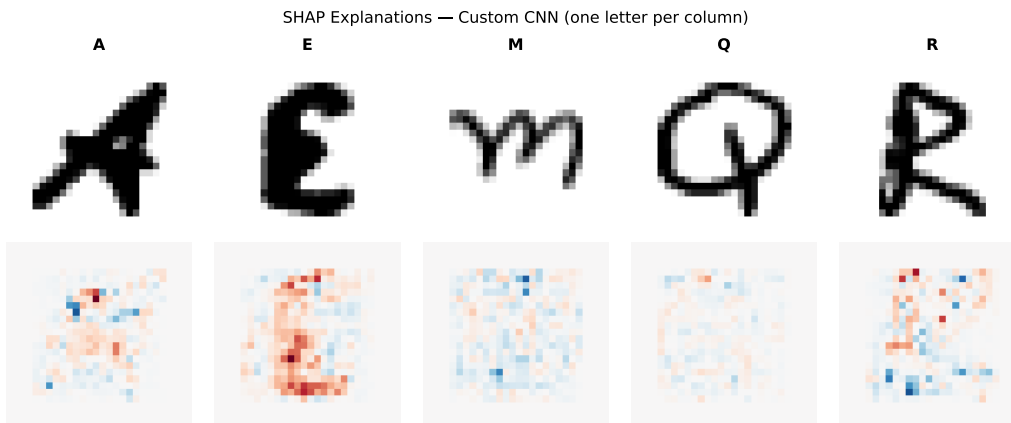


Figure 16: SHAP DeepExplainer attribution maps for five correctly classified letters (A, E, M, Q, R). The top row shows the original images and the bottom row shows the SHAP maps for the true class of each letter. Red pixels are positive contributions that increase the predicted class probability and blue pixels are negative contributions that decrease it. The strongest positive attributions align with the ink stroke regions for all five letters, confirming that the model is using stroke features rather than background pixels.

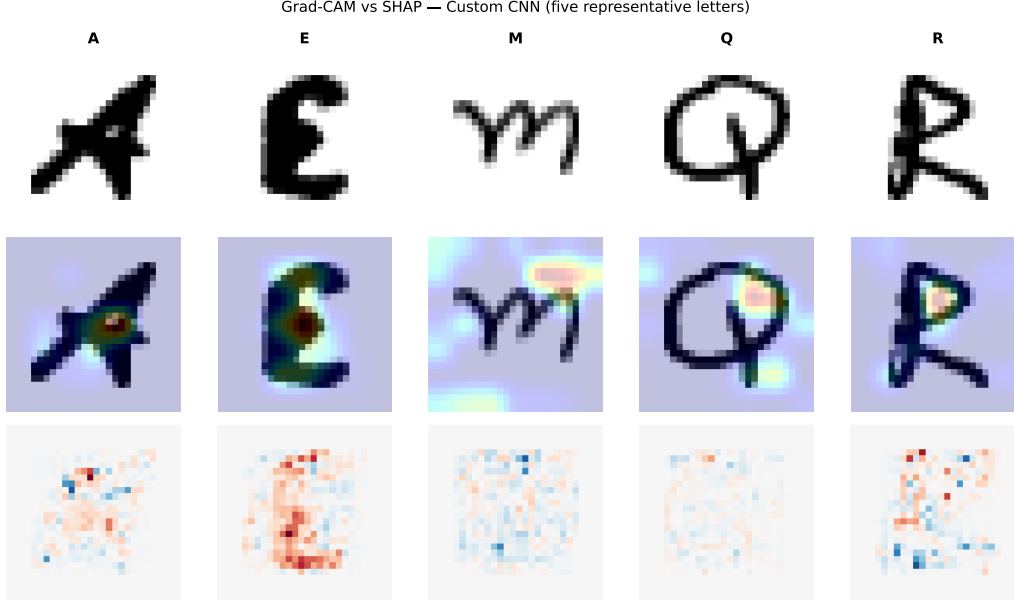


Figure 17: Side-by-side comparison of Grad-CAM (row 2) and SHAP (row 3) explanations for the same five letters (A, E, M, Q, R). Row 1 shows the original test images. Both methods consistently highlight the ink stroke regions as the most important parts of the image, providing convergent evidence that the model is attending to meaningful letter features. Grad-CAM produces smoother coarse heatmaps while SHAP gives sparser pixel-level attributions with both positive and negative values.

Table 7: Comparison of the proposed custom CNN with results from related published studies. Direct numerical comparison is limited by differences in dataset, number of classes, image resolution, and evaluation protocol. XAI = Explainable AI used; EV = External Validation performed.

Study	Dataset	Model	Accuracy	Macro F1	XAI	EV
Baldominos et al. [12]	EMNIST Letters	CNN variants	~0.931	—	✗	✗
Cohen et al. [13]	EMNIST Letters	CNN	~0.854	—	✗	✗
Ahlawat & Choudhary [15]	MNIST Digits	CNN-SVM	~0.986	—	✗	✗
Deng et al. [14]	Custom dataset	Deep CNN	~0.970	—	✗	✗
Chauhan et al. [29]	Multiple scripts	HCR-Net	~0.990	—	✗	✗
Sherpa & Pant [22]	BloodMNIST	CNN ensemble	~0.975	—	✓	✗
Proposed study	A–Z Kaggle	Custom CNN	0.9908	0.9880	✓	✗

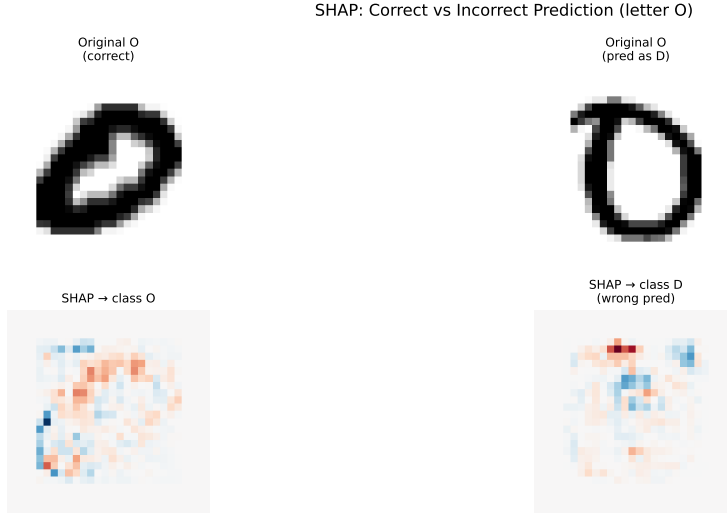


Figure 18: SHAP comparison between a correctly classified letter O (left two columns) and an O that was misclassified as D (right two columns). The top row shows the original images and the bottom row shows the SHAP attributions for the predicted class. For the correct O prediction, positive attribution is spread around the circular loop. For the wrong prediction, the attribution shifts to a vertical edge region on the right side of the image, which resembles the straight side of the letter D and explains why the model was misled.

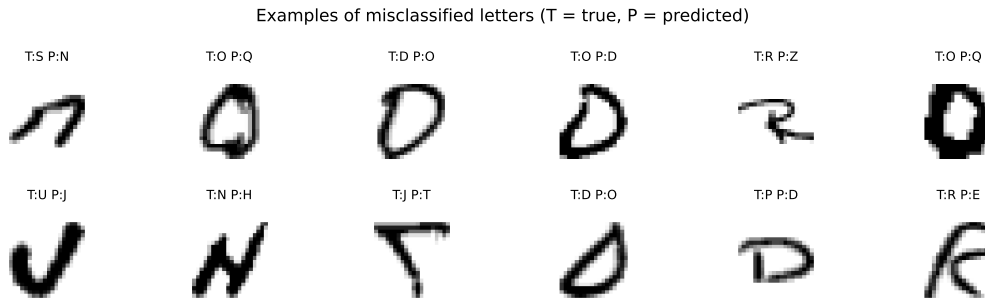


Figure 19: Examples of misclassified letters from the custom CNN test set, showing the true class label (T) and the predicted class label (P) for each image. Most errors occur between visually similar letter pairs such as O and D, confirming that the errors are caused by stroke similarity rather than random model failures.

Table 8: Summary of the five research questions, the evidence used to answer them, the main result, and whether the question was answered. All five questions were answered by the experiments in this study.

RQ	Evidence	Main result	Answered
RQ1	Test accuracy, macro F1, AUC	99.08% accuracy, F1 98.80%, AUC 0.9999	✓
RQ2	Confusion matrix, confused pairs	O→D (64), D→O (32), O→Q (25)	✓
RQ3	Per-class recall for F and I	F recall 0.994, I recall 0.976 after weighting	✓
RQ4	Model comparison table	MobileNetV2 19.59% vs CNN 99.08%	✓
RQ5	Grad-CAM, SHAP, comparison figure	Both methods confirm stroke-region attention	✓